Self-Supervised Similar Community Search Based on Graph Matching Network

Runhuai Chen, Yuxiang Wang, Tianxing Wu, Zhiyuan Yu, Xiaoliang Xu, Xiangyu Ke, and Yuanshi Zheng

***Abstract*—Communities in information networks often appear as cohesive subgraphs, reflecting strong relationships among entities. Given an information network G and a query community C_q, similar community search (SCS) identifies a result community $C_r \subseteq G$ that is similar to C_q in terms of community-level features. SCS has broad applications in social marketing, online recommendation, and other domains. Supervised learning-based methods, such as neural subgraph matching (NSM) and graph alignment (GA), offer flexible learning and efficient inference, showing promising potential. However, they still have certain drawbacks. First, no existing method defines an effective metric to quantify the similarity between C_q and C_r at the community level. Second, GA emphasizes node-wise feature matching, but overlooks structure-wise matching, leading to matching failures when node features are missing. Third, the lack of high-quality human-annotated communities limits the effectiveness of supervised learning. To address these issues, we propose a unified metric for SCS that evaluates community similarity from multiple perspectives, including community cohesiveness, size, and density. We then propose SCS-GMN, an SCS solution based on the Graph Matching Network, which utilizes community similarity-guided self-supervised learning to enhance the cross-graph matching between G and C_q, considering not only node-wise features but also structure-wise features. Moreover, we extend SCS-GMN to large graphs by introducing candidate subgraph generation and a graph-matching transformer. Empirical results over seven real-world datasets show that SCS-GMN efficiently returns C_r with an average community similarity of 96.65% to C_q within 64.5 ms, representing a 14.61% similarity improvement and a 2× inference speedup over suboptimal methods.**

***Index Terms*—Similar community search, graph matching network, graph transformer, information network.**

Received 24 September 2025; revised 20 April 2026; accepted 13 June 2026. Date of publication 29 June 2026; date of current version 8 August 2026. This work was supported in part by the NSFC under Grant 62572162 and Grant 62376058, in part by the Primary R&D Plan of Zhejiang under Grant 2023C03198, in part by the "Pioneer" and "Leading Goose" R&D Program of Zhejiang under Grant 2024C01020, in part by the National Key R&D Program of China under Grant 2026YFE0201600, in part by ZhiShan Young Scholar Program of Southeast University, and in part by the Southeast University Interdisciplinary Research Program for Young Scholars. Recommended for acceptance by R. Akbarinia. *(Corresponding authors: Yuxiang Wang; Tianxing Wu.)*

Runhuai Chen is with Hangzhou Dianzi University, Zhejiang 310005, China, and also with Southeast University, Jiangsu 211102, China.

Yuxiang Wang, Zhiyuan Yu, and Xiaoliang Xu are with Hangzhou Dianzi University, Zhejiang 310005, China (e-mail: lsswyx@hdu.edu.cn).

Tianxing Wu is with Southeast University, Jiangsu 211102, China (e-mail: tianxingwu@seu.edu.cn).

Xiangyu Ke is with Zhejiang University, Zhejiang 310027, China.

Yuanshi Zheng is with Xidian University, Xi'an 710071, China.

Digital Object Identifier 10.1109/TKDE.2026.3707934

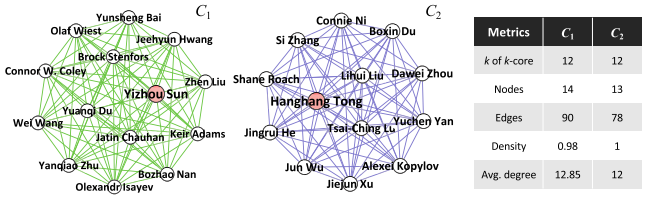
I. Introduction

GRAPHS are extensively used to model real-world information networks across various domains, such as social networks [1], criminal networks [2], and biological networks [3]. Communities, i.e., cohesive substructures modeled as k-core or k-truss, are prevalent in graphs [4]. In graph data analysis, a fundamental topic is *community search* (CS), which focuses on node-driven community engagement. Specifically, it aims to identify a cohesive community C containing a query node q from a graph G, i.e., mapping a query node to its engaged community. For instance, in the DBLP collaboration network, one can employ k-core (a subgraph where each node has a degree $\geq k$) as the community model and establish CS on DBLP to find a cohesive research community for a given query researcher [5]. Fig. 1 (left) illustrates a 12-core community with Prof. Sun (a renowned researcher in graph mining) as the query. It comprises 14 researchers, each maintaining collaborations with at least 12 others. CS has applications in various areas such as event planning [5], [6], advertising [7], expert finding [8], [9], and GraphRAG for retrieval-augmented LLMs [10], [11].

Although CS has attracted significant attention, focusing solely on community engagement remains insufficient in many scenarios. To fully realize its potential in complex applications, such as social marketing and structural genomics, it is critical to incorporate inter-community similarity analysis. This motivates the *similar community search* (SCS) problem studied in this paper, which we briefly introduce below (along with its applications) and formally define in Section II.

Similar community search (SCS): Given a graph G and a query community C_q, SCS identifies a result community $C_r \subseteq G$ similar to C_q. Unlike the classical node-driven CS problem that focuses on mapping a specific query node to its engaged community [12], [13], [14], SCS adopts a graph-driven paradigm, treating the entire C_q as a structural template to retrieve subgraphs preserving similar macroscopic topological patterns and internal cohesion. This positions SCS as a distinct problem rather than an incremental variant of CS. Naturally, SCS and CS may synergize seamlessly: CS can first identify a node's local community, which SCS then uses as a template to discover similar communities, thereby extending community analysis from local node-centric exploration to global structure-aware retrieval. For instance, given Prof. Sun's research community shown in Fig. 1(left) as C_q, SCS may further explore a similar research community C_r in the graph mining area from DBLP, such as Prof. Tong's community in Fig. 1(middle).

Applications: SCS can be applied to many complex applications. (1) *Social marketing.* In social networks, people typically belong to different communities based on a universal human



Metrics	C_1	C_2
k of k-core	12	12
Nodes	14	13
Edges	90	78
Density	0.98	1
Avg. degree	12.85	12

Fig. 1. An example of similar communities in DBLP.

interaction mechanism [15]. By identifying similar communities based on analogous behaviors (such as following behavior or shopping behavior), companies can establish recommendations (such as short videos or products) at entire communities, rather than few individuals [16]. (2) *Structural genomics*. It aims to find hypothetical proteins with similar molecular functions from biological networks [17]. This can be confidently supported by SCS [18], based on the assumption that proteins with similar cohesive structures have similar functions [19], [20]. (3) *Criminal group discovery*. In criminal networks, organized crime groups often exhibit structural similarities [2]. The police can utilize SCS to spot potential crime groups similar to known ones, enabling precise crime enforcement [21], [22]. Moreover, SCS can be integrated with traditional CS for combined applications. For example, one can first use CS to find the research community to which a specific researcher of interest belongs, and then use SCS to obtain its similar research community, thereby facilitating the discovery of potential collaborators. Another example is using CS to identify the criminal group that a suspect belongs to, and then employing SCS to find more similar criminal groups. By integrating them together, the potential value of communities in real-world applications can be further unleashed.

Challenges: Despite its broad applications, SCS has been largely ignored in the literature. In this paper, we aim to bridge this gap by formalizing the SCS problem and addressing the following two key challenges. **First**, *how to effectively measure community similarity from multiple perspectives via a unified metric*? Since a community is fundamentally defined as a cohesive subgraph, a simple strategy is to directly apply standard graph similarity metrics, such as Graph Edit Distance (GED) [23], to measure similarity. However, GED often fails to capture the macro-level structural integrity that defines a community's essence. Consider a k-core community, defined as a maximal subgraph in which each node has at least k neighbors within the subgraph. Removing a single edge incident to a node of degree k increases the GED by only 1. Yet, this minor local change can trigger a *cascading collapse* of the entire community, as the node's removal propagates degree reductions to neighbors, initiating a recursive peeling process that dismantles the k-core structure [24]. This demonstrates that relying solely on one micro-topological metric is insufficient to preserve community coherence. Instead, we resort to exploring other macroscopic metrics to evaluate community similarity. As illustrated in Fig. 1 (right table), two communities are similar across five metrics, including community cohesiveness, size (number of nodes), and density, which are widely used to measure the community quality [25], [26]. Consequently, a key challenge lies in integrating these diverse metrics to achieve a comprehensive, multi-perspective assessment of community similarity. **Additionally**, the entire graph G is typically much larger than the query community C_q, containing numerous dissimilar parts (i.e., noise) to C_q. For example, DBLP contains over 0.3M ($\text{M}{=}10^6$) researchers, much larger than a concentrated research community, which typically consists of only dozens to hundreds of researchers. This raises another challenge: *how to effectively identify a community similar to C_q within a large, noisy graph G*? We next discuss the potential solutions and their limitations, and conclude our solution and contributions.

Potential solutions and limitations: Since node-driven CS only considers which community a query node engages in, it is difficult to be directly extended to the graph-driven SCS. Considering the nature of graph-to-graph mapping in SCS, supervised learning-based *neural subgraph matching* [27], [28], [29], [30] and *graph alignment* [31], [32], [33] are considered potential solutions. Compared to combinatorial subgraph matching algorithms, such as graph isomorphism and graph simulation, the flexible learning and efficient inference capabilities of neural solutions show promising potential [29]. However, **they still have certain drawbacks**. **First**, none of them offer a way to effectively quantify community similarity. Specifically, what constitutes community similarity and how to measure it are both lacking. Consequently, they fail to guide the learning process toward increasing community similarity during training, nor can they quantitatively measure the quality of result community C_r w.r.t. query community C_q. **Second**, graph alignment emphasizes node-wise matching but overlooks the community structure matching. This leads to matching failures when node features (e.g., attributes or vectorized embeddings) are missing, resulting in dissimilar communities. Specifically, it does not adequately consider the significant impact of community structure on the learning model. In other words, the model lacks an understanding of what structure makes a similar community, preventing it from learning representative features for identifying a similar community C_r from a noisy graph G. **Third**, high-quality human-annotated ground-truth communities are often required for supervised learning, but acquiring them is challenging and costly [34]. This highlights the need to reduce reliance on external labeled data to lower learning costs and enhance model scalability.

Our solution and contributions: We briefly outline our solutions to the above limitations and conclude the contributions.

1) Unified community similarity metric: To address the first drawback, in Section II, we first discuss the intuition and necessity of incorporating multiple perspectives to measure community similarity, based on which we propose a unified metric considering three inherent representative community features: cohesiveness, size, and density, to evaluate community similarity between two communities C_i and C_j, denoted by $\texttt{CSIM}(C_i, C_j)$, using *structural similarity index* (SSIM) [35]. Next, we tackle other drawbacks by proposing the *SCS based on Graph Matching Network* called SCS-GMN (Section III-A), which includes two phases: *cross-graph matching* (Section III-B) and *similarity-based self-supervised learning* (Section III-C).

2) Cross-graph matching: To address the second drawback, we establish a matching network on top of two Graph Neural Network (GNN) models, which can additionally capture the structural features of the query community C_q and the entire graph G, beyond node features. The matching network enables the exchange of the obtained structural features, which are sensitive to community cohesiveness, among carefully selected pair-wise nodes from C_q and G. Consequently, the learning process on G can perceive which parts are similar to C_q, effectively avoiding noise interference from dissimilar parts.

It is worth mentioning that when node features are available, structural features can further enhance the learning performance. Conversely, when node features are missing, structural features can to some extent compensate for the performance loss caused by the absence of node features.

3) Similarity-based self-supervised learning: We present a joint loss function based on two easily obtainable self-supervised signals to address the third drawback. The first one comes from the differences in cohesiveness, size, and density between C_r and C_q. Smaller differences lead to higher community similarity $\texttt{CSIM}(C_q, C_r)$. Meanwhile, we design a systematic method based on controllable structural perturbation to generate two similar communities as self-labelled data, denoted by $\langle C_l^*, C_l \rangle$. Then, we take C_l^* as C_q to find a C_r. Since C_l^* is similar to C_l, it is reasonable to take C_l as a label for self-supervised learning to increase $\texttt{CSIM}(C_r, C_l)$, thus increasing $\texttt{CSIM}(C_r, C_l^*)$ as well.

In summary, our main contributions are outlined below.

- We propose a unified metric $\texttt{CSIM}()$ to evaluate community similarity from multiple aspects by considering three representative community-level features, i.e., community cohesiveness, size, and density, using SSIM. Based on this, we formulate the SCS problem, broadening the application of communities in real-world scenarios (Section II).
- We present an end-to-end SCS-GMN model that utilizes community similarity-guided self-supervised learning to enhance the cross-graph matching incorporating both structure-wise and node-wise features, thereby improving the effectiveness of SCS by optimizing the model towards increasing $\texttt{CSIM}(\cdot, \cdot)$ (Section III).
- We extend our SCS-GMN to large-scale graph scenarios by reducing the training-time resource usage through cohesiveness-aware candidate subgraph generation and a graph-matching transformer (Section IV).
- Extensive experiments on eight real-world graphs, including several with millions of nodes and one with over ten million nodes, demonstrate that SCS-GMN attains an average community similarity of 96.65% within 64.5 ms, marking an average 14.61% similarity improvement and 2× inference speedup over suboptimal methods (Section V).

II. Preliminary and Problem

We consider an undirected graph $G = (V, E)$ with a node set V and an edge set E. Given a node $v \in V$, we denote its neighbors and degree by $N(v)$ and $deg(v)$, respectively. When the context has ambiguity, we will clarify the degree in a certain graph as $deg(v, G)$. In Table I, we list the frequently used notations and their explanations.

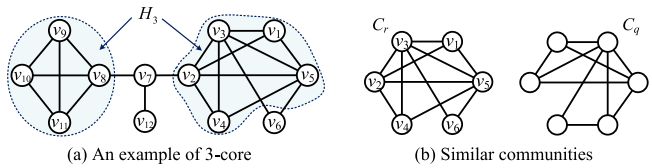
Community model: In community search (CS) literature, models such as k-core [36], [37], k-truss [38], [39], [40], k-clique [41], [42], and k-ECC [43] are often used to depict a community's cohesiveness. Among them, k-core is the most efficient and effective for evaluating cohesiveness [5], [25], [44].

Definition 1 (k-core [36]): Given a graph $G = (V, E)$ and a non-negative integer k, a k-core of G is the largest subgraph $H_k \subseteq G$, such that $\forall v \in H_k$ has a degree at least k, i.e., $deg(v, H_k) \geq k$.

Note that a larger k increases the cohesiveness of H_k. Since a k-core can be a disconnected subgraph, which contradicts the inherent connectivity of communities [45], we follow the same settings of previous studies [5], [44], [45] to **define the connected k-core to represent a community in this paper**.

TABLE I
Frequently Used Notations and Their Explanations

Notations	Explanations
G	undirected graph
V	node set of G
E	edge set of G
$N(v)$	neighbor set of node v
$deg(v, G)$	degree of v in G (shorten as $deg(v)$)
C	community modeled as a connected k-core
C_q	query community
C_r	result community
C_l	self-labelled community
$c(v)$	coreness of node v
$\texttt{Coh}(C)$	cohesiveness of a community C
$\texttt{Den}(C)$	density of a community C
$\texttt{CohSIM}(\cdot, \cdot)$	cohesiveness similarity between communities
$\texttt{SizeSIM}(\cdot, \cdot)$	size similarity between communities
$\texttt{DenSIM}(\cdot, \cdot)$	density similarity between communities
$\texttt{CSIM}(\cdot, \cdot)$/$\texttt{CSIM}()$	community similarity between communities
$\boldsymbol{V}$	$\boldsymbol{V} \in \mathbb{R}^{n \times d}$, node features
$\boldsymbol{W}$	$\boldsymbol{W}^{(l+1)} \in \mathbb{R}^{d^{(l)} \times d^{(l+1)}}$, learnable weight matrix
b	$b^{(l+1)} \in \mathbb{R}^{d^{(l+1)}}$, learnable bias
$\delta(\cdot, \cdot)$	cosine similarity between vectors
$M(u)$	candidate node set
δ_τ	community membership threshold
$\boldsymbol{A}_{V_{\delta_\tau}}$	original adjacency matrix of V_{δ_τ}
$\boldsymbol{A}_{C_r}$	weighted reconstructed adjacency matrix of C_r



(a) An example of 3-core (b) Similar communities

Fig. 2. An example of k-core and SCS.

Definition 2 (Connected k-core [44]): Given a graph $G = (V, E)$ and a non-negative integer k, a connected k-core of G is a connected subgraph $\tilde{H}_k \subseteq H_k \subseteq G$, so that $\forall v \in \tilde{H}_k$, $deg(v, \tilde{H}_k) \geq k$.

Example 1: As shown in Fig. 2(a), the 1-core H_1 is the entire graph and the 2-core H_2 is the subgraph that excludes v_{12}. Both H_1 and H_2 are connected graphs. In contrast, the 3-core H_3 is the largest (disconnected) subgraph, consisting of two components indicated in blue, with each component being a connected 3-core.

Community similarity: Without loss of generality, we denote a community as C, represented by a connected k-core $\tilde{H}_k$. Intuitively, two communities are similar if they exhibit similarity in various community-related features. According to [46], multiple macroscopic structural metrics are available to capture features of individual communities, including explicit ones such as community size, density, and diameter, as well as many implicit features like cohesiveness (e.g., average coreness of all nodes in a graph), triangle participation ratio (TPR), and clustering coefficient (CC). Table II provides brief descriptions and computational complexity of these metrics. To evaluate the similarity between two communities, a straightforward approach would be to compute all pertinent structural metrics for each community and then compare these metrics individually. However, such a comprehensive comparison across all metrics is somehow redundant and not necessary. As claimed in [46], there are strong correlations among these structural metrics, and using a few representative metrics can achieve the same level of effectiveness in similarity measurement. Our preliminary

TABLE II
COMPUTATIONAL COMPLEXITY OF COMMUNITY-RELATED STRUCTURAL METRICS (d DENOTES THE AVERAGE DEGREE OF A GRAPH)

Metrics	Description	Complexity
Size	number of nodes	$O(1)$
Density	ratio of existing to possible edges	$O(1)$
Cohesiveness	average coreness of nodes	$O(\|V\|+\|E\|)$
TPR	fraction of nodes in triangles	$O(\|V\|\cdot d^2)$
CC	clustering coefficient	$O(\|V\|\cdot d^2)$
Diameter	longest shortest path between any pair of nodes	$O(\|V\|\cdot(\|V\|+\|E\|))$

experiments also verified this perspective: we calculated the above metrics for communities in various real-world graphs, such as Cora [47], Citeseer [48], Pubmed [49], Deezer [50], and Facebook [51], and computed the Pearson Correlation Coefficient (PCC) between each pair of metrics. We found that the pairs of metrics, e.g., size and diameter, density and CC, all exhibit strong correlations with PCC > 0.7, as further illustrated in our experimental study (Section V-B). These observations suggest that community similarity can be effectively characterized using a small number of representative structural aspects rather than all possible metrics. Thus, we characterize community similarity from three representative structural aspects, namely cohesiveness, size, and density, which preserve effectiveness while reducing computational cost (see the Complexity column in Table II). We next discuss how to integrate these three metrics appropriately to measure community similarity effectively.

In this paper, we draw on the *structural similarity index measure* (SSIM) [35] from the field of computer vision to integrate multiple distinct structural metrics into a unified community similarity measure. This idea stems from the fact that SSIM has been proven to be a highly effective method for measuring image similarity from multiple image-related metrics, including luminance, contrast, and structure. Analogously, by substituting these image-related metrics with the three community-related structural metrics, we can easily measure how similar two communities are. Thus, we define the community similarity $\texttt{CSIM}(C_i, C_j)$ between two communities C_i and C_j in the form of SSIM as (1), where $\texttt{CohSIM}(C_i, C_j)$, $\texttt{SizeSIM}(C_i, C_j)$, and $\texttt{DenSIM}(C_i, C_j)$ denote cohesiveness, size, and density similarity of two communities, respectively. It is worth mentioning that CSIM adopts a multiplicative SSIM-form because cohesiveness, size, and density represent complementary yet distinct aspects of community structure. A high similarity in one aspect should not compensate for a significant mismatch in another. The multiplicative mechanism enforces a **non-compensatory constraint**: a high CSIM is achieved only when two communities exhibit strong similarity across all three dimensions simultaneously.

$$\texttt{CSIM}(C_i, C_j) = \texttt{CohSIM}(C_i, C_j) \times \texttt{SizeSIM}(C_i, C_j) \times \texttt{DenSIM}(C_i, C_j) \tag{1}$$

We next introduce how to calculate these three sub-item similarities CohSIM, SizeSIM, and DenSIM in detail.

1) Cohesiveness similarity: Given two connected k-core communities C_i and C_j, a simple method to distinguish the cohesiveness of them is to directly compare their k-values. However, it is problematic when they have the same k. For instance, the two communities shown in Fig. 2(b) are both connected 2-cores $\tilde{H}_2$, yet the left one is slightly more cohesive. To solve this, we incorporate the concept of *coreness* to measure the cohesiveness similarity.

Definition 3 (Coreness [45]): Given a graph $G = (V, E)$ and a node $v \in V$, we call the largest k of the k-core that v belongs to as the coreness of v, denoted by $c(v)$.

Intuitively, the more nodes with higher coreness, the more cohesive the community is [5], [44]. So, we define the cohesiveness of a community C as the average coreness of all its members V_C, denoted by $coh(C)$ (2). Recall the two communities in Fig. 2(b), the left one has a $coh(C) = 2.83$, which is larger than that of $coh(C) = 2.66$ for the right one. This indicates that the left community is more cohesive, even though both of them have the same k-value as 2.

$$coh(C) = \frac{\sum_{v \in V_C} c(v)}{|V_C|} \tag{2}$$

We compute the cohesiveness similarity between C_i and C_j as (3), which can be viewed as a sum-of-squares-based normalized formulation that measures the relative closeness between two values, rather than their absolute magnitudes.

$$\texttt{CohSIM}(C_i, C_j) = \frac{2 \cdot coh(C_i) \cdot coh(C_j) + \alpha_1}{coh(C_i)^2 + coh(C_j)^2 + \alpha_1} \tag{3}$$

Besides, it imposes a stricter penalty on magnitude mismatches due to the quadratic denominator, ensuring that high similarity scores are only achieved when community features are proportionally consistent. Here, α_1 is a small constant (not a learnable parameter) to ensure that the denominator is not zero, avoiding numerical instability in special cases [52], [53], [54]. We note that the constants α_2 and α_3 in (4)–(5) serve the same purpose. In this paper, we set $\alpha_1 = \alpha_2 = \alpha_3 = 0.01$.

2) Size similarity: It is evident that communities of similar size (i.e., # nodes) are more alike than those with significant differences [44], [55]. For example, a research group consisting of dozens of people is itself a cohesive research community, typically led by well-known scholars in the field, and is fundamentally different from a research group with only one research fellow and a few students. So, we define size similarity between C_i and C_j as (4).

$$\texttt{SizeSIM}(C_i, C_j) = \frac{2 \cdot |V_{C_i}| \cdot |V_{C_j}| + \alpha_2}{|V_{C_i}|^2 + |V_{C_j}|^2 + \alpha_2} \tag{4}$$

3) Density similarity: Density is often used to evaluate community quality in addition to cohesiveness [9], [44]. It is defined as the internal edge density of the node set [56]. Two communities with comparable density are more alike in structure than those with substantial differences. We evaluate the density similarity as (5), where $den(C) = \frac{2|E_C|}{|V_C| \cdot (|V_C|-1)}$ is the density of a community C.

$$\texttt{DenSIM}(C_i, C_j) = \frac{2 \cdot den(C_i) \cdot den(C_j) + \alpha_3}{den(C_i)^2 + den(C_j)^2 + \alpha_3} \tag{5}$$

We substitute (3)–(5) into (1) to obtain the community similarity, denoted as $\texttt{CSIM}(C_i, C_j)$, as shown in (6). Following the form of SSIM [35], $\texttt{CSIM}(C_i, C_j)$ retains an advantage meaning in that a high community similarity is produced only when all three sub-item similarities are similar.

$$\texttt{CSIM}(C_i, C_j) = \frac{2 \cdot coh(C_i) \cdot coh(C_j) + \alpha_1}{coh(C_i)^2 + coh(C_j)^2 + \alpha_1}$$

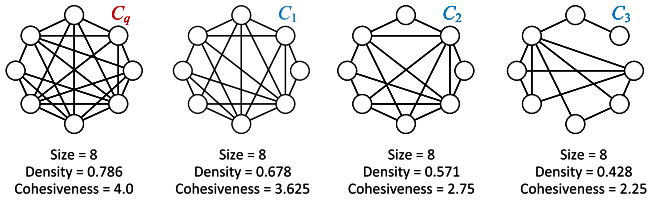


Fig. 3. Communities with different similar metrics to a query community C_q.

$$\times \frac{2 \cdot |V_{C_i}| \cdot |V_{C_j}| + \alpha_2}{|V_{C_i}|^2 + |V_{C_j}|^2 + \alpha_2} \times \frac{2 \cdot den(C_i) \cdot den(C_j) + \alpha_3}{den(C_i)^2 + den(C_j)^2 + \alpha_3} \tag{6}$$

Note that our $\texttt{CSIM}(C_i, C_j)$ adopts the multiplicative SSIM-form formulation of each similarity component does not introduce exponent weights. The reason is two-fold. First, all three component similarities (CohSIM, SizeSIM, and DenSIM) are rigorously normalized to the same bounded range of [0,1]. Since they share a unified scale, there is no discrepancy in magnitude that requires correction via exponent weights. Second, within a multiplicative framework of normalized values, exponent weights ($\neq 1$) function primarily as tuners for non-compensatory sensitivity. An exponent weight >1 makes the metric *hyper-sensitive*, causing the total score to plummet drastically even with minor deviations in that specific component. An exponent weight <1 makes the metric *insensitive*, allowing significant mismatches to be tolerated. Since there is no theoretical or empirical prior suggesting that one aspect systematically outweighs the others across diverse scenarios, we posit that cohesiveness, size, and density are equally critical orthogonal dimensions. Therefore, adopting unitary exponent weight $=1$ is the unbiased and robust strategy, avoiding arbitrary sensitivity biases.

Similar Community Search (SCS) Problem: Given a target graph G and a query community C_q, SCS identifies a result community $C_r \subseteq G$ that has the largest community similarity to C_q (7).

$$C_r = \arg\max_{C_i \subseteq G} \texttt{CSIM}(C_i, C_q) \tag{7}$$

Example 2: Given a query community C_q in Fig. 3 (left), we present three potential result communities $\{C_1, C_2, C_3\}$ that have the same size but exhibit gradually decreasing densities and cohesiveness. Intuitively, C_1 should be the most similar to C_q, while C_3 should be the least similar one. The community similarity measure can effectively reflect these explicit differences. By applying $\texttt{CSIM}(\cdot, \cdot)$ (6), we can quantify such difference between each $C_i \in \{C_1, C_2, C_3\}$ and C_q. Specifically, C_1 has the most similar structural features to C_q, resulting in a community similarity of 98%. In contrast, C_2 and C_3 have similarities of 87% and 71%, respectively. Therefore, the SCS problem should return C_1 as the result community due to its highest community similarity to C_q.

Remarks: First, it is worth noting that the query community C_q can come from outside the target graph G, allowing it to accept out-of-distribution inputs, which greatly enhances the applicability of SCS in real-world scenarios. Second, SCS can also be extended to k-truss-based communities, which impose stronger structural constraints by requiring that every edge resides in at least $k-2$ triangles (i.e., adjacent nodes share a minimum of $k-2$ common neighbors) [40]. In this case, we only modify the cohesiveness component CohSIM in CSIM by substituting average node coreness with average edge trussness, while SizeSIM and DenSIM remain unchanged. More details are given in Section III-C (the Remarks part). Third, As noted in [46], *triangle participation ratio* (TPR), *clustering coefficient* (CC), and *diameter* have implicit correlation with the three structural features that we used in defining community similarity, therefore maintaining the similarity of cohesiveness, size, and density helps preserve the similarity of TPR, CC, and diameter. In Section V-C, we provide experimental results w.r.t. the similarity of TPR, CC, and diameter between C_q and C_r.

III. The Proposed SCS-GMN

In this section, we present the proposed *Similar Community Search based on Graph Matching Network* (SCS-GMN).

A. Model Overview

Fig. 4 illustrates the pipeline of SCS-GMN, which comprises two phases: *cross-graph matching* (Section III-B) and *similarity-based self-supervised learning* (Section III-C). The objective of cross-graph matching is to perceive which parts of graph G are potentially similar to the query community C_q and generate a feasible result community C_r accordingly. While the objective of similarity-based self-supervised learning is to fine-tune the model parameters using self-supervised losses, enhancing the first phase's ability to generate a C_r with greater community similarity $\texttt{CSIM}(C_r, C_q)$ to C_q.

Cross-graph matching: Since the community similarity is reflected in structural features, an intuitive idea is to learn the structural features of G and C_q separately, then enhance mutual perception of similar parts through cross-graph feature interaction. To achieve this, we apply two GNN models on C_q and G to preserve their structural features in their own node embeddings (**step** ① in Fig. 4). Next, we filter promising nodes from G as matching candidates for each node in C_q, generating a set of potentially matched node pairs (**step** ②). Then, we perform cross-graph feature interaction between matched nodes, enhancing their respective node embeddings using the same GNN model (**steps** ③-④). As a result, the matched nodes are likely to exhibit more similar embeddings compared to other nodes. We first predict a soft community vector over target nodes using the enhanced node embeddings, from which thresholding yields the discrete result community C_r for inference. Since this hard-threshold induced subgraph is discrete and non-differentiable, for training we further construct a differentiable weighted adjacency matrix $\boldsymbol{A}_{C_r}$ on the selected nodes to support end-to-end self-supervised learning (**step** ⑥).

Similarity-based self-supervised learning: In this phase, we establish a similarity-guided joint loss based on two self-supervised losses. (1) Since our community similarity $\texttt{CSIM}()$ is based on three independent similarity metrics, it makes sense to design a structural self-supervised loss that reflects the differences in these metrics between the reconstructed C_r and the input C_q. The smaller the differences in each metric, the greater the $\texttt{CSIM}(C_r, C_q)$. (2) We present a systematic method based on controllable structural perturbation to automatically generate a similar community C_l to C_q, serving as self-labelled data. Intuitively, if C_r shows greater community similarity to C_l, this also encourages C_r to be similar to C_q in terms of $\texttt{CSIM}()$. Thus, we introduce a label-based self-supervised loss derived from the

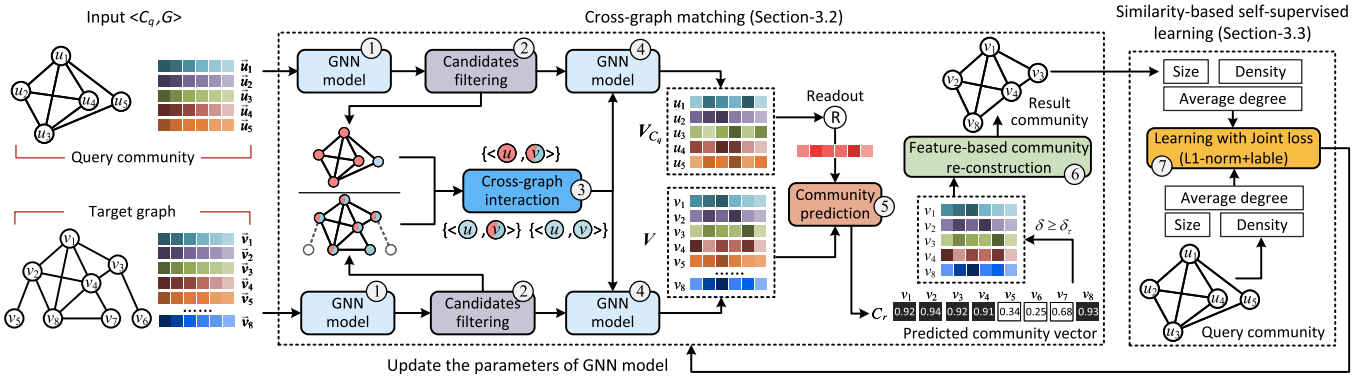


Fig. 4. The pipeline of SCS-GMN: cross-graph matching (steps ①-②) and similarity-based self-supervision (step ⑦).

differences between the reconstructed C_r and the self-labelled community C_l (**step ⑦**).

B. Cross-Graph Matching

Given a target graph $G = (V, E)$ and a query community C_q, we discuss the cross-graph matching in detail.

① *GNN model:* A GNN model serves as a cornerstone of SCS-GMN. Various GNN models are available to choose, such as GCN [57], GraphSAGE [58], and GAT [59]. We use a GNN model to obtain the embeddings of all nodes from V and V_{C_q}, denoted by $\boldsymbol{V}$ and $\boldsymbol{V}_{C_q}$. For each node $v \in V$ ($u \in V_{C_q}$), it has a representation $\vec{v} \in \boldsymbol{V}$ ($\vec{u} \in \boldsymbol{V}_{C_q}$). We provide the general intra-layer processes of a GNN model as follows, where $v_n \in N(v) \cup v$ indicates a neighbor of v.

$$\vec{v}^{(l+1)} = \texttt{Dr}(\phi(\texttt{AGG}(\vec{v}_n^{(l)}\boldsymbol{W}^{(l+1)} + b^{(l+1)}))) \tag{8}$$

Note that, $\vec{v}^{(l+1)} \in \mathbb{R}^{d^{(l+1)}}$ is the representation of v with $d^{(l+1)}$ dimensions in the $(l+1)$-th layer, $\vec{u}^{(l)} \in \mathbb{R}^{d^{(l)}}$ is the representation of each $u \in N(v)$ from the l-th layer, and the input $\vec{v}^{(0)} \in \mathbb{R}^{d}$ is the initial representation of v. Besides, $\boldsymbol{W}^{(l+1)} \in \mathbb{R}^{d^{(l)} \times d^{(l+1)}}$ and $b^{(l+1)} \in \mathbb{R}^{d^{(l+1)}}$ are learnable weights and bias. Moreover, $\texttt{AGG}(\cdot)$ is an aggregation function such as AVG, SUM, MAX, or MIN, $\phi(\cdot)$ is the non-linear activation function, e.g., $\texttt{ReLU}(\cdot)$, and $\texttt{Dr}(\cdot)$ is the dropout method [60] to relieve overfitting in neural networks. The graph topology features will be embedded into the representation $\vec{v}^{(l+1)}$ after several epochs. Because of the ease of use and simplicity of GCN, we adopt the classical GCN in our default implementation. In Section V-F, we study the effect of various GNN models on SCS-GMN's performance, by replacing GCN with GAT and GraphSAGE.

② *Candidates filtering:* Since the goal of SCS-GMN is to identify a similar result community $C_r \subseteq G$ to a given C_q, a pivotal aspect involves the interaction of structural features between potential matched nodes in G and C_q, which helps accurately identify the similar parts of G and C_q by learning structural features from each other. The key is to find potential matched nodes from G and C_q. Given a node $u \in V_{C_q}$, we filter a set of candidate nodes $\{v_1, \ldots, v_n\} \in V$ for matching with u based on the following observation.

Theorem 1: Given a node $u \in V_{C_q}$ with a coreness $c(u)$, all nodes from G that have a degree $deg(v, G) < c(u)$ are not possible to exist in a connected k-core community C with a higher $k \geq c(u)$, i.e., for $\forall v \in V_C$, its coreness satisfies $c(v) < c(u)$.

Proof: Since a connected k-core community C is a subgraph of G, we have $deg(v, C) \leq deg(v, G)$. So, if we have $deg(v, G) < c(u)$, then $deg(v, C) < c(u)$ holds. This implies that the largest k-value of the connected k-core community containing v would be at most $c(u) - 1$, thereby we have $c(v) \leq c(u) - 1 < c(u)$. □

Based on Theorem 1, a community C consisting of nodes with a degree $deg(v, G) < c(u)$ (where $u \in V_{C_q}$) cannot achieve the same level of community cohesiveness $\texttt{Coh}()$ as C_q. This is because $\texttt{Coh}(C) < \texttt{Coh}(C_q)$ when every node $v \in V_C$ has a coreness $c(v) < c(u)$. This prompts us to pay more attention to those nodes with a degree $deg(v, G) \geq c(u)$ as candidates for matching with u. Given a query community C_q, we first apply a classical core-decomposition method [61] to compute the coreness of each node $u \in V_{C_q}$. Then, we determine the potential matched nodes for u as $M(u) = \{v | deg(v, G) \geq c(u)\}$. Recall the C_q shown in Fig. 4, nodes $\{u_1 \sim u_4\}$ have the same coreness of 3. Each node $u \in \{u_1 \sim u_4\}$ has the same $M(u) = \{v_1, v_2, v_3, v_4, v_8\}$ because these nodes have a degree ≥ 3, resulting in $4 \times 5 = 20$ potential matching node pairs, denoted by $\{\langle u, v\rangle\}$. While considering u_5 in C_q, since its coreness $c(u_5) = 2$, we have $M(u_5) = \{v_1, v_2, v_3, v_4, v_7, v_8\}$ as they have a degree ≥ 2. In Fig. 4 (middle), we use different colors to illustrate the potential matching node pairs. In this example, nodes v_5 and v_6 are excluded from any pairs, because their degrees are smaller than the coreness of any node in C_q, indicating their dissimilarity in terms of community cohesiveness. Thus, it is not necessary to exchange their structural features with nodes from C_q during cross-graph interaction.

③ *Cross-graph interaction:* Given the matched nodes $\{\langle u, v\rangle\}$, we design a cross-graph feature interaction method to enhance the similarity of their representations. More precisely, we expect that a node u's representation $\vec{u}$ is more similar to its matched node v than to other nodes. To achieve this, for each node u from $\{\langle u, v\rangle\}$, we adopt (9) to aggregate the features of all u's matched nodes, where $\delta(\cdot, \cdot)$ is the cosine similarity between two vectors.

$$\vec{u} = \sum \delta(\vec{u}, \vec{v}) \cdot \vec{v} + \vec{u} \tag{9}$$

We clarify that the interaction is bidirectional, for each v from $\{\langle u, v\rangle\}$ we take the same way for features aggregation.

Example 3: Considering the node u_5 from the query community C_q in Fig. 4, its matched nodes are $M(u_5) = \{v_1, v_2, v_3, v_4, v_7, v_8\}$, so that we update $\vec{u}_5 = \sum_{v \in M(u_5)} \delta(\vec{u}_5, \vec{v}) \cdot \vec{v} + \vec{u}_5$. In contrast, considering the node v_7 from the target graph G. Since

it is involved in just one matched node pair $\langle u_5, v_7\rangle$, we update $\vec{v}_7 = \delta(\vec{u}_5, \vec{v}_7) \cdot \vec{u}_5 + \vec{v}_7$.

④ *Embeddings enhancement via the same GNN model:* We next take the updated node embeddings from the cross-graph interaction step as the input of the same GNN models that are applied in step ① to refine the node embeddings of G and C_q. From a macro-perspective, this ensures that nodes in one graph can acquire both the structural features within its own graph and the structural features of similar nodes in the other graph, thus enhancing the cross-graph node representation capability.

⑤ *Community prediction:* Given the node embeddings $\boldsymbol{V}_{C_q}$ and $\boldsymbol{V}$ obtained after step ④, we predict the result community C_r as follows. First, we use an *average readout function* on $\boldsymbol{V}_{C_q}$ to get a representation of C_q, denoted by $R(\boldsymbol{V}_{C_q}) = \sum_{\vec{u}\in\boldsymbol{V}_{C_q}} \vec{u}/|V_{C_q}|$. Then, we compute the cosine similarity $\delta(\vec{v}, R(\boldsymbol{V}_{C_q}))$ as the prediction of the community involvement of v. The greater the $\delta(\cdot)$, the more the probability of v being part of C_r. We maintain a predicted community vector $\vec{C}_r$, where each dimension has a value of $\delta(\vec{v}, R(\boldsymbol{V}_{C_q}))$. At inference, nodes with scores no smaller than δ_τ are selected, and the induced subgraph on these nodes is returned as the discrete result community C_r.

⑥ *Feature-based community reconstruction:* A direct way to obtain the discrete result community from $\vec{C}_r$ is to set a threshold δ_τ and take the induced subgraph on nodes with $\delta(\vec{v}, R(\boldsymbol{V}_{C_q})) \geq \delta_\tau$ as C_r. However, this induced-subgraph construction is non-differentiable and therefore not suitable for end-to-end learning. To address this issue, we retain this hard-threshold induced subgraph as the final inference output, while additionally constructing a differentiable weighted adjacency matrix $\boldsymbol{A}_{C_r}$ on the selected nodes, based on the assumption that nodes with similar representations are likely to be directly connected [62]. Specifically, we reconstruct $\boldsymbol{A}_{C_r}$ by (10).

$$\boldsymbol{A}_{C_r} = \boldsymbol{V}_{\delta_\tau} \cdot \boldsymbol{V}_{\delta_\tau} \cdot \boldsymbol{A}_{V_{\delta_\tau}} \tag{10}$$

Here, $\boldsymbol{A}_{C_r}$ is the weighted adjacency matrix of the constructed C_r, where each element $a_{ij} \in \boldsymbol{A}_{C_r}$ indicates the probability that two nodes v_i and v_j have an edge in C_r, $\boldsymbol{V}_{\delta_\tau}$ denotes the representations of nodes V_{δ_τ} with $\delta(\vec{v}, R(\boldsymbol{V}_{C_q})) \geq \delta_\tau$, and $\boldsymbol{A}_{V_{\delta_\tau}} \subseteq \boldsymbol{A}_G$ is the adjacency matrix of V_{δ_τ} derived from the original graph's adjacency matrix. In this way, $\boldsymbol{A}_{C_r}$ can be viewed as a differentiable function of node representations obtained by the GNN model, yielding a matrix that remains tied to the predicted community structure and supports differentiable computation of the community structural metrics used in the subsequent self-supervised losses.

C. Similarity-Based Self-Supervised Learning

We now discuss how to design an appropriate loss to complete the end-to-end learning using the functional $\boldsymbol{A}_{C_r}$.

⑦ *Learning with joint loss:* We present two distinct self-supervised losses and combine them as a joint loss for model training. The first loss evaluates the differences in three community-related metrics between the differentiable reconstruction $\boldsymbol{A}_{C_r}$ of the predicted community and the query community C_q. The second loss is derived from the automatically generated self-labels that take into account community similarity.

1) Structural self-supervised loss: Considering the community similarity $\texttt{CSIM}(C_r, C_q)$ defined in (1), it is established atop cohesiveness, size, and density similarities. The cohesiveness similarity $\texttt{CohSIM}(C_r, C_q)$ relates to each node's coreness $c(\cdot)$. Since $c(\cdot)$ cannot be directly calculated from $\boldsymbol{A}_{C_r}$ via vector operations, i.e., it is non-differentiable, we utilize a differentiable node degree $deg(\cdot)$ instead of $c(\cdot)$, as it serves as an upper bound of $c()$), and alternatively take into account the average degree of a community to design the L1-norm loss as (11). This choice is motivated by the observation that average degree and average coreness often exhibit a strong positive correlation [63], [64]. We also demonstrate this empirically through the correlation analysis in Section V-B, where we evaluate their correlation across communities with various k values sampled from Cora, Pubmed, and DBLP datasets, yielding an average PCC of 0.942.

$$\mathcal{L}_{\text{coh}} = \frac{1}{m}\sum_{i=1}^{m} |\overline{deg}(C_{r_i}) - \overline{deg}(C_{q_i})| \tag{11}$$

Here, m is the batch size (i.e., m queries $\{C_{q_1}, \ldots, C_{q_m}\}$) and $\overline{deg}(C_{r_i}) = \frac{1}{|V_{C_{r_i}}|}\sum_{j=1}^{|V_{C_{r_i}}|}\sum_{k=1}^{|V_{C_{r_i}}|} \boldsymbol{A}^{j,k}_{C_{r_i}}$ is the differentiable average degree of C_{r_i}, computed from $\boldsymbol{A}_{C_{r_i}}$, where $\boldsymbol{A}^{j,k}_{C_{r_i}}$ is the element at the j-th row and k-th column. It is worth mentioning that the influence of coreness $c(\cdot)$ is considered in step ② to obtain matched nodes for cross-graph feature interaction.

For size similarity $\texttt{SizeSIM}(C_r, C_q)$, greater similarity is indicated by a smaller difference between $|V_{C_r}|$ and $|V_{C_q}|$. Thus, we design the following L1-norm loss, where $|V_{C_{r_i}}| = \texttt{Tr}(\boldsymbol{A}_{C_{r_i}})$ ($\texttt{Tr}(\cdot)$ is the differentiable trace operation).

$$\mathcal{L}_{\text{size}} = \frac{1}{m}\sum_{i=1}^{m} ||V_{C_{r_i}}| - |V_{C_{q_i}}|| \tag{12}$$

Considering the density similarity $\texttt{DenSIM}(C_r, C_q)$, the smaller the difference between $den(C_r)$ and $den(C_q)$, the greater the similarity. Therefore, we design the following L1-norm loss, where $den(C_{r_i})$ is the differentiable density obtained from $\boldsymbol{A}_{C_{r_i}}$.

$$\mathcal{L}_{\text{den}} = \frac{1}{m}\sum_{i=1}^{m} |den(C_{r_i}) - den(C_{q_i})| \tag{13}$$

$$den(C_{r_i}) = \frac{\frac{1}{2}\sum_{j=1}^{|V_{C_{r_i}}|}\sum_{k=1}^{|V_{C_{r_i}}|} \boldsymbol{A}^{j,k}_{C_{r_i}}}{\texttt{Tr}(\boldsymbol{A}_{C_{r_i}})(\texttt{Tr}(\boldsymbol{A}_{C_{r_i}}) - 1)} \tag{14}$$

2) Label-based self-supervised loss: Intuitively, an effective strategy to enhance learning is to explicitly inform the model which communities are similar. This involves using easily obtainable similar communities as self-supervised signals to improve learning quality. We generate similar communities systematically by considering community similarity as follows. Given a target graph G, we extract a set of connected k-core communities from G, denoted by $\mathcal{C} = \{C_{l_1}, \ldots, C_{l_n}\}$. For each C_{l_i}, we apply a certain degree of perturbation (e.g., remove edges, nodes, and introduce noise to original node features) to obtain another community $C^*_{l_i}$. If the community similarity $\texttt{CSIM}(C^*_{l_i}, C_{l_i}) \geq \texttt{CSIM}_\tau$, we consider $\langle C^*_{l_i}, C_{l_i}\rangle$ as a self-labelled data. $\texttt{CSIM}_\tau$ is a hyperparameter used to control the quality of self-labelled data. Next, we take $C^*_{l_i}$ as a query C_{q_i} to find a result C_{r_i} and design a label-based loss as (15), by comparing the community vectors of C_{r_i} and C_{l_i}, where $\vec{C}_{r_i}$ is obtained at Step ⑤ (community prediction) and C_{l_i}

is represented as a binary 0-1 vector, showing the community involvement of each node.

$$\mathcal{L}_{\text{label}} = \frac{1}{m}\sum_{i=1}^{m}(\vec{C}_{r_i} - \vec{C}_{l_i})^2 \tag{15}$$

Thus, we expect to improve the similarity between C_{r_i} and $C^*_{l_i}$ by approximating the similarity between C_{r_i} and C_{l_i}.

3) Joint loss: Given the above losses, we combine them as a joint loss for self-supervised learning, where α is used to balance the structural self-supervised loss and label-based self-supervised loss.

$$\mathcal{L}_{\text{joint}} = \alpha(\mathcal{L}_{\text{coh}} + \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{den}}) + \mathcal{L}_{\text{label}} \tag{16}$$

Remarks: Besides k-core, SCS-GMN can also be adapted to k-truss-based communities. A k-truss is a subgraph in which every edge e_{uv} is contained in at least $k-2$ triangles, that is, the two endpoints u and v have at least $k-2$ common neighbors. Accordingly, extending the proposed framework to k-truss mainly requires replacing the cohesiveness-related quantities while keeping the overall architecture unchanged. As discussed above for `CSIM`, for k-truss-based communities, the average coreness used in `CohSIM` is replaced by the average edge trussness of a community, whereas `SizeSIM` and `DenSIM` are retained. For candidates filtering, instead of using the coreness of query nodes together with the degree of target nodes as in the k-core case, the filtering criterion is changed to a truss-based one. Specifically, the trussness of query edges is used to identify target edges with sufficiently large support as potential matches, and the incident nodes of these retained edges are then taken as candidate nodes for subsequent cross-graph matching. For self-supervised learning, since edge trussness cannot be directly computed in a differentiable manner from the reconstructed weighted adjacency matrix, we replace the degree-based differentiable quantity used in the k-core case with differentiable average edge support to capture truss-based cohesiveness. Moreover, we generate truss-based self-labelled data instead of core-based self-labelled data for self-supervised learning, following the same procedure described in Section III-C. In this way, the proposed framework can be naturally adapted from k-core-based communities to k-truss-based communities.

IV. Extension to Large Graphs

Benefiting from a small number of model parameters and cross-graph feature interaction with candidate filtering, SCS-GMN can be trained on large-scale graphs. However, global training on such graphs implies lower training efficiency. Additionally, since SCS-GMN relies on GCN or GCN-like methods for intra-graph message passing, it encounters two major challenges when applied to large-scale graphs. First, it requires the entire graph's adjacency matrix as input, leading to significant memory overhead with a space complexity of $O(N \times N)$, where N is the number of nodes. Second, the limited receptive field of GNNs makes it difficult to capture long-range dependencies, causing important information from distant nodes to be diluted or even lost during training. Therefore, to better adapt to SCS in large-scale graph scenarios, we introduce two simple modifications to the original SCS-GMN while keeping all other components unchanged. On the one hand, we propose a cohesiveness-aware candidate subgraph generation strategy that leverages the inherent structural information of the query community to effectively reduce the input size of the target graph. On the other hand, we design a graph matching transformer to enhance the representation learning capability of SCS-GMN. The architecture of the modified SCS-GMN is illustrated in Fig. 5, where the newly introduced modules are highlighted in blue.

1) Cohesiveness-aware candidate subgraph generation: When the target graph is large in scale, directly using the entire graph as input not only incurs significant memory overhead but also introduces numerous irrelevant and noisy regions that are dissimilar to the query community C_q, thereby severely degrading representation learning performance and training efficiency. These issues indicate that, in the context of SCS on large-scale graphs, reducing the size of the input graph is both feasible and necessary. This inspires us to present a cohesiveness-aware candidate subgraph generation method to reduce the target graph G into a smaller subgraph G', and take G' as the input to SCS-GMN instead of G. Specifically, when a connected k-core is given as a query community C_q, we restrict our attention to nodes in G whose degrees are at least k, as only they are likely to form communities with such a level of internal cohesiveness of k. To achieve this, we first compute the internal degree of each node v in C_q, i.e., $deg(v, G)$, and obtain its cohesiveness level $k = \min_{v \in C_q} \deg(v, C_q)$. Then, we traverse the target graph G to select all nodes whose degree is no less than k and extract the induced subgraph on these nodes as the candidate subgraph. In practice, the statistics obtained from our experimental study show that the generated candidate subgraphs contain **16456.84** nodes on average, which is only **0.2688%** of the size of the original target graphs, showing that this step is effective to generate a quite small subgraph that is beneficial for the next step of graph matching transformer.

Example 4: Given a query community C_q shown in Fig. 5 (left top), we first compute the internal degrees of all nodes in C_q as $\{u_1 : 4, u_2 : 3, u_3 : 4, u_4 : 3, u_5 : 2\}$, and obtain the minimum degree $k = 2$, indicating that C_q is a connected 2-core. Subsequently, we adopt $k = 2$ as a threshold to filter nodes within the target graph G in Fig. 5 (left bottom), resulting in a candidate node set $\{v_1, v_2, v_3, v_4, v_7, v_8\}$. The subgraph induced by these nodes is taken as the candidate subgraph G'. Notably, this subgraph preserves a level of cohesiveness similar to that of C_q, thereby providing a reasonable and high-quality search space for subsequent matching.

2) Graph matching transformer: In traditional GNN frameworks, node representation learning relies on the adjacency matrix to enable intra-graph message passing. However, this approach incurs substantial memory overhead when applied to large-scale graphs. Therefore, there is an urgent need for a more effective node representation learning method tailored to large-scale graphs. In response, we propose using a Graph Transformer (GT) to replace the GNN model (step ①) for enhancing representation learning. In our implementation, we employ a Graphormer-style full self-attention mechanism on G' following [65], while incorporating structural information through positional encodings. This design enables precise attention computation on the reduced subgraph generated in step (1) rather than on the entire target graph G. Specifically, we treat all nodes in G' as a long sequence and directly learn the global correlations among nodes through a full self-attention mechanism. Compared to traditional GNNs, it does not rely on the adjacency matrix as input and avoids the limitations caused by stacking multiple layers, thereby enabling more direct modeling of long-range dependencies within the reduced graph input.

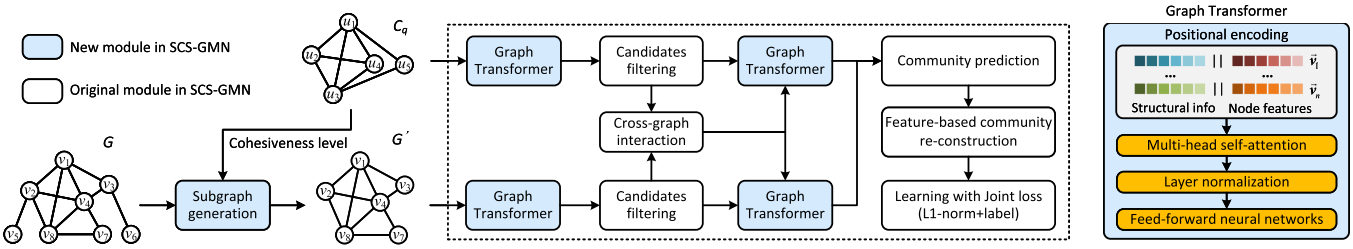


Fig. 5. The detailed architecture of the modified SCS-GMN.

Since the attention computation is performed on G' instead of G, its quadratic cost depends on the size of the candidate subgraph rather than the full target graph size.

Fig. 5(right) illustrates the details of Graph Transformer, which comprises positional encoding, multi-head self-attention, and feed-forward neural networks. We will next introduce the functionality of each component in detail.

Positional encoding (PE): Since GT does not explicitly depend on the graph adjacency matrix, structural positional information of nodes is incorporated to enhance the model's awareness of connectivity among nodes. Specifically, given the initial node feature $\vec{x}_v$ (e.g., attributes or vectorized embeddings), we construct a structural information vector $\vec{p}_v$, which includes normalized values of node degree, h-index, coreness, and clustering coefficient (CC). This structural information vector $\vec{p}_v$ is concatenated with the node features to embed the spatial positional information of nodes. Formally, this can be expressed as follows.

$$\vec{p}_v = [deg(v) \,||\, h\text{-index}(v) \,||\, c(v) \,||\, \text{CC}(v)] \tag{17}$$

$$\vec{v}^{(0)} = [\vec{x}_v \,||\, \vec{p}_v] \tag{18}$$

Here, h-index(v) and CC(v) represent the h-index and clustering coefficient of node v, while $deg(v)$ and $c(v)$ are degree and cohesiveness of v that we formally defined in Section II.

Multi-head self-attention (MHA): After constructing the input features, node representations are iteratively updated through multiple layers of the Graph Transformer to integrate structural and attribute information from relevant nodes. Therefore, the node representation at the l-th layer, denoted as $\vec{v}^{(l)}$, would be processed via MHA as below:

$$MHA(\vec{v}^{(l)}) = Concat(head_1, \ldots, head_h)\boldsymbol{W}^{(l)} \quad , \tag{19}$$

where $\boldsymbol{W}^{(l)} \in \mathbb{R}^{hd_m^{(l+1)} \times d_m^{(l+1)}}$ is a learnable parameter used to aggregate the outputs from multiple attention heads and $head_i$ denotes the attention computation of the i-th head, i.e.,

$$head_i = Attention(\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v}) = \text{softmax}\left(\frac{\boldsymbol{q}\boldsymbol{k}^\top}{\sqrt{d_m^{(l+1)}}}\right)\boldsymbol{v} \quad . \tag{20}$$

Here, the query, key, and value vectors are computed as $\boldsymbol{q} = \vec{v}^{(l)}\boldsymbol{W}_i^{\boldsymbol{q}}$, $\boldsymbol{k} = \vec{v}^{(l)}\boldsymbol{W}_i^{\boldsymbol{k}}$, and $\boldsymbol{v} = \vec{v}^{(l)}\boldsymbol{W}_i^{\boldsymbol{v}}$ respectively, where $\boldsymbol{W}_i^q \in \mathbb{R}^{d_m^{(l)} \times d_m^{(l+1)}}$, $\boldsymbol{W}_i^k \in \mathbb{R}^{d_m^{(l)} \times d_m^{(l+1)}}$, $\boldsymbol{W}_i^v \in \mathbb{R}^{d_m^{(l)} \times d_m^{(l+1)}}$ are all learnable weights to project $\vec{v}^{(l)}$ into different matrices. Through the full attention mechanism, we capture the global structural information of the community, rather than being limited to the local neighborhood features of individual nodes.

Feed-forward neural networks (FFN): Finally, a FFN is applied to the output of the multi-head self-attention layer to enhance the model's nonlinear representation capacity. FFN consists of a two-layer multilayer perceptron (MLP) with ReLU activation, where $LN(\cdot)$ is the layer normalization.

$$\begin{aligned} \vec{v}^{(l+1)} &= MHA(LN(\vec{v}^{(l)})) + \vec{v}^{(l)} \\ \vec{v}^{(l+1)} &= FFN(LN(\vec{v}^{(l+1)})) + \vec{v}^{(l+1)} \end{aligned} \tag{21}$$

It is worth noting that the proposed Graph Transformer replaces the original GNN model in SCS-GMN to enhance node representation capability in large-scale graph scenarios. In addition, to further strengthen the connection between the query community and potential similar regions in the target graph for improved performance, we also incorporate the cross-graph interaction described in steps ② and ③ between GT layers, and employ the joint loss function from step ⑦ to train the model toward the similar community search objective.

In our experimental study Section V-I, we analyze the scalability of the improved SCS-GMN on large-scale graphs by extracting target graphs with node scales ranging from millions to tens of millions from the ogbn-papers100M [66] dataset to validate both training efficiency and effectiveness.

V. Experiments

We evaluate (1) effectiveness (Section V-C), (2) efficiency (Section V-D), (3) offline overhead (Section V-E), (4) effect of various GNN models (Section V-F), (5) ablation study (Section V-G), (6) parameters sensitivity (Section V-H), (7) scalability (Section V-I), and (8) case study (Section V-J) to answer the below questions. Unless otherwise specified, all experiments were conducted on a Linux server equipped with an NVIDIA RTX A6000 GPU (48 GB), using Python 3.8.18, PyTorch 1.13.1 with CUDA 11.6, DGL 0.9.1, and NetworkX 3.1. Our code and dataset is available at [67].

Q1. How do different community structural metrics correlate on representative datasets, and what do these correlations imply for the design of `CSIM`? (Section V-B)

Q2. How does the effectiveness and efficiency of the proposed SCS-GMN compared to other methods? (Section V-C-Section V-D)

Q3. What is the offline training and storage overhead of the proposed SCS-GMN and other comparing methods? (Section V-E)

Q4. What is the effect of various GNN models on SCS-GMN's effectiveness? (Section V-F)

Q5. What is the effect of two types of self-supervised losses on SCS-GMN's effectiveness? (Section V-G)

TABLE III
DATASET STATISTICS

Datasets ↓	# Nodes	# Edges	d_{avg}	$k_{\max}$
Cora [47]	2,708	5,429	3.9	4
Citeseer [48]	3,312	4,732	2.7	7
Pubmed [49]	19,717	44,338	4.5	10
Deezer [50]	28,281	92,752	6.6	12
Facebook [51]	22,470	171,002	15.2	56
DBLP [68]	317,080	1,049,866	6.6	113
Reddit [58]	232,965	30,515,502	223.6	213
ogbn-papers100M [66]	111,059,956	1,615,685,872	29.1	59

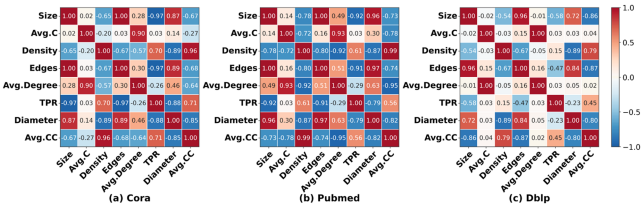


Fig. 6. Pearson correlation heatmaps of community structural metrics on Cora, Pubmed, and DBLP.

Q6. What is the effects of $\texttt{CSIM}_\tau$ in step ⑦ and the number of GNN layers on the effectiveness of SCS-GMN? (Section V-H)

Q7. How is the scalability of SCS-GMN on large-scale graphs in terms of training overhead and performance? (Section V-I)

Q8. What are differences between communities returned by various methods in real-world datasets? (case study in Section V-J)

A. Experimental Setup

Datasets: Table III provides statistics of seven real-world datasets, including the number of nodes and edges, average degree (d_{avg}), and the largest k of k-core ($k_{\max}$) in a graph. Cora [47], Citeseer [48], and Pubmed [49] are well-known citation networks. Deezer [50], Reddit [58], and Facebook [51] are representative social networks. DBLP [68] is an academic collaboration network. Ogbn-papers100M [66] is a large-scale citation network, which we use exclusively in subsection V-I to evaluate the scalability of SCS-GMN in large graph scenarios. It is worth noting that for datasets with node attributes, we incorporate them into training as follows: (1) If the attributes are provided as categorical fields, we convert them into one-hot vectors and form the node features as the input of SCS-GMN. (2) If vectorized attribute features are provided, we directly use them as the node features.

Queries: We generate query communities as follows. (1) We perform k-core decomposition on G to obtain all possible k values in G. (2) We extract k-cores with the top-20% k values as the source communities. (3) We perturb these source communities as query communities, following the instructions in the part of "Label-based self-supervised loss", Section III-C. The same set of generated query communities is used for all compared methods on each dataset. We run 500 queries for each dataset and provide the average results in terms of the following metrics.

Comparing methods: We compared our SCS-GMN with two categories of possible solutions: neural subgraph matching (1-6) and graph alignment (7-10). For neural subgraph matching, we compared with (1) **NeuroMatch** [29], which designs a loss function based on the partial order property of graph representations to reflect the relationship between graphs and subgraphs. (2) **Sub-GMN** [30], a method that models node-level similarity between two graphs using neural tensor networks. (3) **AED-Net** [69], which introduces edge matching constraints during node representation learning to emphasize isomorphic matching relationships. (4) **IA-NGM** [70], which node features with predicted matches to amplify their discrepancy from ground-truth, thus enhancing matching accuracy. (5) **IA-SSGM** [71], it performs node swapping based on predicted matches to create pseudo-samples, leveraging contrastive learning to improve the robustness of graph matching and (6) **StableGM** [72], which treats NSM as a contrastive learning task with hard negative sampling and contrastive loss for training. For graph alignment, we compared with (7) **GCNAlign** [31], which leverages GCN to aggregate neighborhood information for enhancing entity representations and its variant (8) **GATAlign** [31], using GAT to capture neighborhood information (9) **GAEA** [73], which considers the role of relation alignment in graph alignment and (10) **DNCN** [74], a method that performs contrastive learning by generating new views through perturbations of the original graph. For the compared baselines, we follow the default or recommended hyperparameter settings and training protocols reported in their original papers, including the original model-selection or early-stopping rules when applicable, and do not perform extra task-specific retuning for the SCS setting beyond the necessary adaptation for output conversion.

Metrics: We run 500 queries for each dataset and provide the average results of the following metrics. For effectiveness, we report the community similarity $\texttt{CSIM}()$ (1) and the average coreness, size, and density of C_q and C_r. The smaller the differences between C_q and C_r on these metrics, the more similar C_r is to C_q. For efficiency, we report the online inference time (millisecond), offline training time (hour), training memory usage (GB), and model size (MB).

Parameters: Default hyperparameters are configured as: learning rate $= 0.01$, epochs $= 200$, dropout ratio $= 0.5$, batch size $= 10$, and GCN layers $= 2$. We set $\texttt{CSIM}_\tau = 0.9$ in step ⑦. In (16), we set $\alpha = 0.01$ for all experiments to balance the structural self-supervised loss and the label-based self-supervised loss.

Remark: To better present the experimental study, we highlight the top-3 results in **bold**, underline, and *italics*, respectively, and use "o.o.m" to indicate out of memory issue.

B. Correlation Analysis of Structural Metrics

To further analyze the relationships among different community structural metrics, we visualize the Pearson correlation matrices on three representative datasets with different scales and network characteristics, namely Cora, Pubmed, and DBLP, as shown in Fig. 6. The results show that these metrics are not independent, but exhibit clear correlation patterns across datasets. In particular, several metric pairs show strong positive correlations, such as size and number of edges (*Edges*), average coreness(*Avg.C*) and average degree (*Avg.Degree*), as well as density and average clustering coefficient (*Avg.CC*), indicating that some metrics reflect similar structural tendencies of communities. In particular, for average coreness and average degree, the average PCC over Cora, Pubmed, and DBLP is about 0.942, confirming a robust positive correlation across representative

TABLE IV
EFFECTIVENESS (LEFT, CSIM(C_q, C_r) (%)) AND EFFICIENCY RESULTS (RIGHT, INFERENCE TIME IN MILLISECOND)

Methods ↓	Cora		Citeseer		Pubmed		Deezer		Facebook		DBLP		Reddit	
	CSIM	Time	CSIM	Time	CSIM	Time	CSIM	Time	CSIM	Time	CSIM	Time	CSIM	Time
SCS-GMN	**97.16**	*59.30*	**97.55**	73.30	**98.32**	**48.52**	**98.67**	**57.68**	**94.54**	**54.89**	**97.96**	**35.62**	**92.41**	**121.86**
NeuroMatch [29]	68.05	3.66e+6	*77.44*	1.08e+6	66.41	4.32e+6	76.62	3.66e+6	65.99	4.24e+6	81.78	*8.72e+6*	73.53	2.61e+9
Sub-GMN [30]	47.98	**27.72**	49.80	**48.36**	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
AEDNet [69]	41.62	299.60	63.51	420.21	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
StableGM [72]	26.02	1362.80	62.00	1362.80	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-SSGM [71]	31.35	168.30	63.26	163.07	50.48	3067.69	11.29	4035.29	27.86	*347.89*	o.o.m	o.o.m	o.o.m	o.o.m
IA-NGM [70]	60.71	422.28	76.99	440.81	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GCNAlign [31]	*82.28*	53.6	60.04	*74.60*	91.81	5943.04	*84.45*	*647.12*	63.09	1097.36	o.o.m	o.o.m	o.o.m	o.o.m
GATAlign [31]	86.41	394	90.33	378.52	*91.79*	7153.47	84.46	2719.02	*65.09*	2755.06	o.o.m	o.o.m	o.o.m	o.o.m
DNCN [74]	68.13	165.46	66.79	232.65	53.77	*2340.51*	52.43	6552.43	50.16	6625.71	*43.07*	8.52e+4	o.o.m	o.o.m
GAEA [73]	63.99	142.67	38.63	260.75	45.46	179.79	67.52	107.99	61.61	177.67	o.o.m	o.o.m	o.o.m	o.o.m

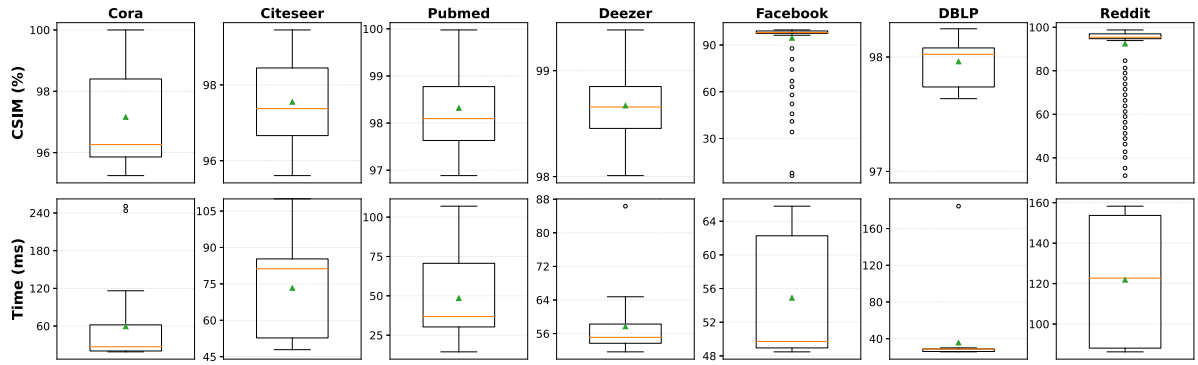


Fig. 7. Boxplots of CSIM (top) and per-query inference time (bottom) for SCS-GMN across datasets.

TABLE V
COMMUNITY DENSITY (DENS), AVERAGE CORENESS (AVG. C), AND SIZE OF C_q AND C_r RETURNED BY VARIOUS METHODS

Methods ↓	Cora			Citeseer			Pubmed			Deezer			Facebook			DBLP			Reddit		
	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size
Query C_q	0.67	4.00	18.90	0.40	6.39	29.20	0.0198	8.59	694.13	0.06	9.90	259.83	0.99	55.19	59.86	0.99	100.50	103.80	0.16	122.22	943.3
SCS-GMN	0.77	**3.75**	19.01	0.45	**5.82**	25.20	0.0210	**8.38**	673.89	**0.06**	**9.20**	274.37	**0.99**	**44.42**	*77.49*	**0.99**	**91.55**	**92.55**	**0.15**	**87.01**	**1125.7**
NeuroMatch	*0.48*	1.75	23.63	0.29	*4.86*	16.22	**0.0205**	4.06	395.03	**0.06**	*5.74*	160.39	0.96	29.11	119.71	**0.99**	64.13	65.36	0.18	72.18	514.9
Sub-GMN	0.17	1.06	5.02	0.15	1.70	110.05	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
AEDNet	0.26	2.61	6.60	0.20	5.49	57.93	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
StableGM	0.06	0.42	**18.90**	**0.44**	3.67	12.97	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-SSGM	0.21	1.14	**18.90**	0.20	3.06	**29.20**	0.005	2.67	**694.13**	0.008	1.48	**259.83**	0.33	13.19	**59.86**	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-NGM	0.33	1.88	**18.90**	0.27	3.52	**29.20**	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GCNAlign	0.45	*2.55*	**18.90**	0.32	2.74	16.35	0.0214	*6.25*	530.93	0.04	6.30	**259.83**	0.51	26.23	**59.86**	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GATAlign	0.45	2.00	*15.79*	*0.34*	4.26	*23.29*	*0.0184*	6.44	520.59	0.04	6.30	**259.83**	0.52	*27.09*	**59.86**	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
DNCN	**0.72**	2.13	9.53	0.26	2.96	36.24	0.012	3.15	468.39	*0.028*	4.00	*224.85*	0.42	21.93	57.77	*0.61*	*33.66*	*55.80*	o.o.m	o.o.m	o.o.m
GAEA	0.39	1.59	13.91	0.21	1.78	18.69	0.008	2.82	*592.55*	0.04	4.42	183.89	*0.67*	23.67	38.91	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m

datasets of different scales. Meanwhile, several metric pairs show clear negative correlations, such as density and diameter, suggesting that different metrics may characterize community structure from different directions.

These results provide empirical support for the design of CSIM. On the one hand, the strong correlations among some metrics indicate that it is unnecessary to include all community statistics simultaneously when measuring community similarity. On the other hand, the selected metrics in CSIM, i.e., **cohesiveness**, **size**, and **density**, capture different essential aspects of community structure, including internal cohesiveness, scale, and compactness. Therefore, using these three representative metrics is reasonable for characterizing community similarity in a compact and effective manner.

C. Effectiveness Evaluation

General effectiveness results: (1) Table IV (left column of each method) shows the CSIM() of all comparing methods. *SCS-GMN achieves an average CSIM() of 96.65% across all datasets, marking an average 14.61% improvement over suboptimal methods.* This is because we measure community similarity from multiple aspects and train SCS-GMN towards increasing CSIM() through a similarity-based joint loss. In addition, Fig. 7 (top) further presents the boxplots of CSIM across queries for SCS-GMN. The results show that CSIM is concentrated in a high-similarity range on most datasets, while a few low-score outliers appear on Facebook and Reddit. Nevertheless, the median CSIM remains high across datasets and is consistent with the average results reported in Table IV. (2) Table V shows the detailed density, average coreness, and size of the query community C_q and result communities C_r returned by various methods. Since Reddit is the most dense dataset (with over 30M edges for only 0.23M nodes), most methods encounter o.o.m issues. Only SCS-GMN and NeuroMatch can run on the Reddit dataset. Smaller differences between C_q and C_r in a single metric indicate greater similarity in this aspect. In most cases, SCS-GMN shows smaller differences (top-2

TABLE VI
EFFECTIVENESS RESULTS OF SCS-GMN ACROSS DIFFERENT k

Metrics ↓	Citeseer			Pubmed				Deezer			
	$k=6$	$k=7$	Mean	$k=8$	$k=9$	$k=10$	Mean	$k=10$	$k=11$	$k=12$	Mean
CSIM	92.61	98.06	95.34	99.32	99.88	99.37	99.53	83.89	99.2	96.37	93.15
Dens (C_q)	0.34	0.57	0.46	0.02	0.04	0.13	0.06	0.25	0.65	0.71	0.54
Dens (C_r)	0.4	0.63	0.51	0.02	0.04	0.13	0.06	0.33	0.68	0.65	0.56
Avg.C (C_q)	6.01	7	6.51	8.71	9.23	10	9.32	10.36	11.42	12	11.26
Avg.C (C_r)	5.84	6.26	6.05	8.88	9.26	9.28	9.14	9.62	10.83	11.15	10.53
Size (C_q)	25	16	20.5	621	414	123	386	101.24	25.12	23.64	50
Size (C_r)	22.49	14.68	18.59	656.4	433.4	114.9	401.6	93.44	23.64	25.7	47.59

TABLE VII
EFFECTIVENESS RESULTS W.R.T. TPR, CC, AND DIAMETER

Datasets ↓	TPR		CC		Diameter	
	Query C_q	SCS-GMN	Query C_q	SCS-GMN	Query C_q	SCS-GMN
Cora	0.99	0.99	0.61	0.78	2.85	2.85
Citeseer	1.0	0.99	0.49	0.53	2.55	2.5
Pubmed	0.98	0.88	0.19	0.19	7.0	8.0
Deezer	1.0	0.97	0.32	0.32	5.0	6.0
Facebook	1.0	1.0	0.95	0.98	2.25	2.0
DBLP	1.0	1.0	1.0	1.0	2.0	1.0
Reddit	1.0	1.0	0.19	0.2	2.0	3.0

results) to that of C_q in all the three metrics. For example, in Deezer, SCS-GMN returns a C_r that is most similar to C_q in community cohesiveness and density, and second-most similar in size (with a relative error of only 5.6%), achieving a better $\texttt{CSIM}() = 98.67\%$ than others.

Effectiveness results for different k: It is worth noting that Tables IV(left column) and V show the average results for query communities with various k values. While in this test, we provide detailed results grouped by different k values (see Table VI), as well as the average result of all listed k values (the last column of each dataset). Due to the page limits, we only report the results of the top-3 k-values for only three datasets. We note that our model performs well for each k value. Specifically, we achieved good results across different k values for density, average coreness, and size with small differences between C_q and C_r, yielding a high community similarity CSIM().

Effectiveness results w.r.t. other metrics: In Table VII, we show the results of our SCS-GMN under three additional metrics of Triangle Participation Ratio (TPR), Clustering Coefficient (CC), and Diameter. These metrics are often used to measure the community's quality. From the presented results, we found that the C_r returned by our SCS-GMN also exhibits a relatively high degree of similarity to C_q in these metrics. This demonstrates that learning features of community cohesiveness, density, and size, would help preserve the structural features of other related metrics to a certain extent.

D. Efficiency Evaluation

Table IV(right column) shows the average online inference time across queries for all methods. For SCS-GMN, this time includes candidate filtering, model forward inference, and output community decoding, while excluding file I/O and one-time offline preprocessing. SCS-GMN achieves comparable efficiency with Sub-GMN and GCNAlign, while outperforming others on small graphs due to its localized feature interactions among a small number of matched nodes. In larger graphs, Sub-GMN suffers from the o.o.m issue due to its large parameter volume, and GCNAlign suffers from inefficient global node matching across the entire graph. In contrast, SCS-GMN remains stable as graph size increases and achieves an average 2× speedup. This can be attributed to the lightweight feature interactions rather than interactions involving the entire graph. Fig. 7 (bottom) further supplements the average runtime results in Table IV by showing the per-query inference-time distribution of SCS-GMN. The runtime is relatively concentrated on most datasets, while a few high-latency outliers can be observed on Cora and DBLP. Overall, the median runtime follows the same trend as the average inference time reported in Table IV.

E. Offline Training and Storage Overhead

Table VIII reports the offline training overhead, including training time and memory usage (left and middle columns), and model storage overhead (right column), i.e., model size. GATAlign requires the most training time due to its reliance on GAT, which involves additional attention calculations. SCS-GMN achieves a comparable training efficiency with the most efficient IA-GMN and GCNAlign for small graphs. For larger graphs, SCS-GMN is more efficient as it only involves two 2-layer GCNs and a lightweight graph matching network. It is worth noting that SCS-GMN and NeuroMatch are the only two that can support the most dense graph Reddit. In terms of runtime memory usage, SCS-GMN's overhead is acceptable compared with others. Besides, SCS-GMN has a modest storage overhead compared to competitors, as it only needs to store a small number of parameters from the two 2-layer GCN models for C_q and G.

F. Effect of Various GNN Models

Evaluate SCS-GMN integrated with several representative GNN backbones, including GCN [57] (our default setting), GAT [59], and GraphSAGE [58]. Table IX shows the effect of GNN models on effectiveness (CSIM) and efficiency (inference and training time) of SCS-GMN, while Table X shows the detailed results w.r.t. three separate metrics. Note that SCS-GMN with GCN achieves the best overall performance in terms of both effectiveness and efficiency, yielding an average community similarity (CSIM) of 96.66%, an inference time of 64.45 ms, and a training time of 0.95 hours. While SCS-GMN with GAT offers superior effectiveness (CSIM of 87.57%) compared to GraphSAGE (CSIM of 83.29%), its attention mechanism incurs higher computational costs. Conversely, although SCS-GMN with GraphSAGE is more efficient than GAT (e.g., 3.3× faster in online inference), its neighborhood sampling strategy may omit critical structural information, thereby weakening the learned representations.

G. Ablation Study

Effect of joint loss: To verify the effect of joint loss, we conduct an ablation study on variants of SCS-GMN with different losses. Table XII shows the CSIM() over all datasets. The first row shows the results using the joint loss $\mathcal{L}_{\text{joint}} = \alpha\mathcal{L}_{\text{structure}} + \mathcal{L}_{\text{label}}$, where $\mathcal{L}_{\text{structure}} = \mathcal{L}_{\text{coh}} + \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{den}}$. The second and third rows show the results using label-based loss and structural loss. From the results in the first three rows, we found that both losses are beneficial for SCS, and the best performance is achieved when using both of them. Moreover, we study the effect of $\mathcal{L}_{\text{coh}}$, $\mathcal{L}_{\text{size}}$, and $\mathcal{L}_{\text{den}}$ by removing each one from $\mathcal{L}_{\text{joint}}$. Note that, each loss is useful for SCS, and the best result is achieved when using them together.

Effect of cross-graph matching: To verify the effect of cross-graph matching, we replace the cross-graph matching component (the middle part of Fig. 4, Section III-B) with other comparing methods, and train the model using the $\mathcal{L}_{\text{joint}}$ as an

TABLE VIII
OFFLINE TRAINING TIME (LEFT, HOUR), MEMORY USAGE (MIDDLE, GB), AND MODEL SIZE (RIGHT, MB)

Methods ↓	Cora			Citeseer			Pubmed			Deezer			Facebook			DBLP			Reddit		
	Time	Mem	Size	Time	Mem	Size	Time	Mem	Size	Time	Mem	Size	Time	Mem	Size	Time	Mem	Size	Time	Mem	Size
SCS-GMN	*0.28*	*1.38*	4.1	*0.31*	1.48	8.6	**0.61**	**1.76**	2.3	**0.51**	**3.24**	*10.0*	**0.54**	**2.91**	*11.0*	**0.46**	7.15	2.0	**3.94**	17.15	2.5
NeuroMatch	6.32	4.15	**0.3**	5.31	4.15	**0.3**	6.18	4.15	**0.3**	7.21	4.15	**0.3**	7.45	4.15	**0.3**	*89.34*	**4.15**	**0.3**	137.28	**4.15**	**0.3**
Sub-GMN	0.56	4.39	3.9	0.50	3.16	*5.0*	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
AEDNet	0.71	*1.38*	4.0	0.45	*1.38*	5.4	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
StableGM	1.42	2.91	112.0	1.47	4.77	111.2	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-SSGM	1.27	1.55	2.6	1.29	1.64	2.6	*2.90*	10.3	*2.6*	1.80	18.73	2.6	2.04	24.82	2.6	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-NGM	**0.09**	3.59	147.2	**0.13**	5.06	147.2	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GCNAlign	0.13	0.33	*2.8*	0.18	**0.34**	5.6	1.99	*4.31*	17.0	*3.73*	*10.31*	25.9	*4.45*	*6.31*	20.9	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GATAlign	4.44	0.91	2.9	4.96	1.16	5.6	45.64	8.71	17.0	80.82	16.77	26.8	57.42	8.77	21.8	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
DNCN	1.37	1.71	353.1	3.48	4.93	2355.2	6.95	5.26	3112.9	28.31	11.47	3420.2	26.62	14.15	3891.2	36.75	*15.71*	*74.30*	o.o.m	o.o.m	o.o.m
GAEA	3.82	1.43	9.3	4.96	1.45	8.8	6.90	10.86	65.0	16.41	21.42	122.0	60.67	30.51	180.0	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m

TABLE IX
EFFECTIVENESS ($\texttt{CSIM}(C_q, C_r)$ (%)) AND EFFICIENCY (INFERENCE TIME (LEFT, MILLISECONDS) AND OFFLINE TRAINING TIME (RIGHT, HOURS)) RESULTS WITH VARIOUS GNN MODELS

Methods ↓	Cora			Citeseer			Pubmed			Deezer			Facebook			DBLP			Reddit		
	CSIM	Inf.	Train.	CSIM	Inf.	Train.	CSIM	Inf.	Train.	CSIM	Inf.	Train.	CSIM	Inf.	Train.	CSIM	Inf.	Train.	CSIM	Inf.	Train.
SCS-GMN (**GCN**)	**97.16**	**59.30**	**0.28**	**97.55**	*73.30*	0.31	**98.32**	**48.52**	**0.61**	**98.67**	**57.68**	**0.51**	**94.54**	**54.89**	**0.54**	**97.96**	**35.62**	**0.46**	**92.41**	121.86	**3.94**
SCS-GMN (**GAT**)	*92.09*	*73.27*	0.40	91.44	72.41	**0.27**	84.64	*155.2*	0.97	79.02	*196.63*	0.63	*84.54*	*177.82*	0.73	92.06	*272.78*	*0.93*	89.19	*537.64*	*4.88*
SCS-GMN (**GraphSAGE**)	93.47	63.13	*0.50*	*81.84*	**55.85**	*0.34*	*79.49*	54.62	*1.55*	*69.67*	76.66	*0.87*	86.47	65.47	*1.06*	86.96	49.64	0.71	*85.11*	**74.54**	4.49

TABLE X
COMMUNITY DENSITY (DENS.), AVERAGE CORENESS (AVG.C), AND SIZE OF THE QUERY COMMUNITY C_q AND RETURNED COMMUNITY C_r WITH VARIOUS GNN MODELS

Methods ↓	Cora			Citeseer			Pubmed			Deezer			Facebook			DBLP			Reddit		
	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size	Dens	Avg.C	Size
Query C_q	0.67	4.00	18.90	0.40	6.39	29.20	0.0198	8.59	694.13	0.06	9.90	259.83	0.99	55.19	59.86	0.99	100.50	103.80	0.16	122.22	943.3
SCS-GMN (**GCN**)	*0.77*	**3.75**	**19.01**	**0.45**	5.82	**25.20**	**0.0210**	**8.38**	673.89	**0.06**	**9.20**	**274.37**	**0.99**	44.42	**77.49**	**0.99**	**91.55**	**92.55**	0.15	**87.01**	**1125.7**
SCS-GMN (**GAT**)	**0.73**	*3.35*	*16.67*	**0.45**	**5.87**	21.22	*0.009*	*6.62*	*734.69*	0.05	*6.74*	*190.85*	**0.99**	*39.12*	*40.39*	**0.99**	*76.7*	*77.7*	**0.16**	173.57	1295.35
SCS-GMN (**GraphSAGE**)	0.75	3.49	18.6	0.48	*5.34*	*16.06*	0.014	6.74	**681.37**	*0.04*	6.77	233.66	**0.99**	**58.62**	41.28	**0.99**	89	90	0.17	*190.11*	*1338.59*

TABLE XI
ABLATION STUDY: CROSS-GRAPH MATCHING (CSIM() (%))

Datasets →	Cora	Citeseer	Pubmed	Deezer	Facebook	DBLP	Reddit
SCS-GMN	**97.16**	**97.55**	**98.32**	**98.67**	**94.54**	**97.96**	**92.41**
NeuroMatch + $\mathcal{L}_{\text{joint}}$	75.95	82.17	79.85	80.96	82.27	91.40	83.95
Sub-GMN + $\mathcal{L}_{\text{joint}}$	54.64	49.80	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
AEDNet + $\mathcal{L}_{\text{joint}}$	42.99	69.85	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
StableGM + $\mathcal{L}_{\text{joint}}$	33.80	74.31	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
IA-SSGM + $\mathcal{L}_{\text{joint}}$	47.76	79.52	62.16	47.32	32.63	o.o.m	o.o.m
IA-NGM + $\mathcal{L}_{\text{joint}}$	66.83	78.67	o.o.m	o.o.m	o.o.m	o.o.m	o.o.m
GCNAlign + $\mathcal{L}_{\text{joint}}$	84.53	77.75	94.49	92.83	89.12	o.o.m	o.o.m
GATAlign + $\mathcal{L}_{\text{joint}}$	88.22	94.98	94.33	92.70	88.69	o.o.m	o.o.m
DNCN + $\mathcal{L}_{\text{joint}}$	71.97	78.14	62.84	66.35	53.05	o.o.m	o.o.m
GAEA + $\mathcal{L}_{\text{joint}}$	71.75	42.42	51.43	68.15	78.31	o.o.m	o.o.m

TABLE XII
ABLATION STUDY: SELF-SUPERVISED LOSSES (CSIM() (%))

Datasets →	Cora	Citeseer	Pubmed	Deezer	Facebook	DBLP	Reddit
$\mathcal{L}_{\text{joint}}$	**97.16**	**97.55**	**98.32**	**98.67**	**94.54**	**86.43**	**92.41**
$\mathcal{L}_{\text{label}}$	92.61	93.76	58.24	77.32	73.46	61.07	36.75
$\mathcal{L}_{\text{structure}}$	59.90	61.94	71.49	35.84	74.65	52.80	26.16
$\mathcal{L}_{\text{joint}} \setminus \mathcal{L}_{\text{den}}$	89.37	95.15	79.76	86.65	87.04	64.72	49.10
$\mathcal{L}_{\text{joint}} \setminus \mathcal{L}_{\text{coh}}$	87.97	96.87	87.96	82.19	87.96	63.92	77.32
$\mathcal{L}_{\text{joint}} \setminus \mathcal{L}_{\text{size}}$	61.75	97.01	78.47	80.02	78.47	61.67	39.71

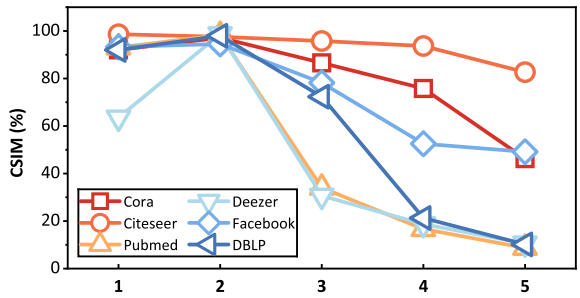


Fig. 8. Effect of GNN layers ($l \in [1, 5]$) on SCS-GMN's effectiveness.

additional loss. Table XI shows the results. We found that using $\mathcal{L}_{\text{joint}}$ can greatly increase the CSIM for all comparing methods compared to the original results shown in Table IV. Additionally, SCS-GMN still outperforms the enhanced comparing methods, highlighting the effectiveness of the cross-graph matching component of SCS-GMN.

H. Parameter Sensitivity

Effect of the number of GNN layers: During the intra-graph message passing (step ①, step ④), we adopt GCN as the backbone of SCS-GMN to learn node embeddings. Naturally, the number of GCN layers will affect the preference of node embeddings for structural information, where shallow layers make nodes focus more on local neighborhood features whereas deeper layers extend the receptive field. Therefore, we demonstrate the effect of GCN layers on SCS-GMN's performance in Fig. 8. We can find that the best results are achieved only at $l = 2$, which indicates that under this setting, SCS-GMN achieves an optimal balance between global and local information aggregation. This is because when the number of layers is small ($l = 1$), the model suffers from limited expressive power due to its narrow receptive field, failing to capture macro community structure. Conversely, when the number of layers is large ($l > 2$), model's performance also decreases due to over-smoothing issues. Besides, in Fig. 9, we summarize the relationship between the number of GCN layers and the training overhead, showing that the training time gradually increases as the number of layers grows.

Effect of $\texttt{CSIM}_\tau$: In the similarity-based self-supervised learning (step ⑦), we introduce a similarity threshold $\texttt{CSIM}_\tau$ for the generation of labelled communities $\langle C_l^*, C_l \rangle$. We show $\texttt{CSIM}_\tau$'s effect on SCS-GMN's effectiveness in Fig. 10. Note

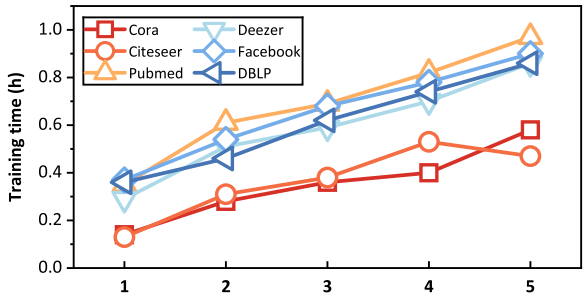


Fig. 9. Effect of GNN layers ($l \in [1, 5]$) on SCS-GMN's training overhead.

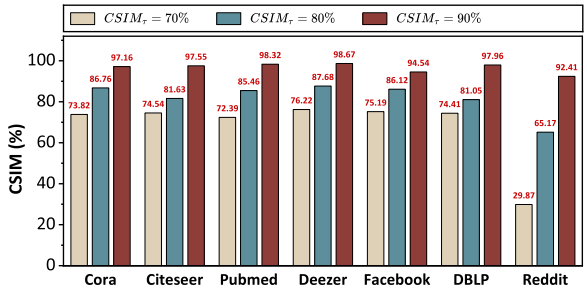


Fig. 10. Effect of $\texttt{CSIM}_\tau$ on SCS-GMN's online inference effectiveness.

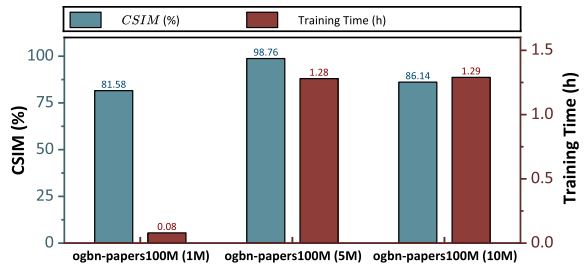


Fig. 11. Training time and online effectiveness of SCS-GMN on large graphs.

that, $\texttt{CSIM}()$ increases as $\texttt{CSIM}_\tau$ increases. This is because we use C_l^* as a query to find C_r and utilize C_l as a self-supervised signal to enhance the training (i.e., making C_r similar to C_l), it's reasonable that the larger the similarity between C_l^* and C_l, the larger the similarity between C_r and C_l^*. For instance, the average $\texttt{CSIM}()$ achieved for $\texttt{CSIM}_\tau = 70\%, 80\%$, and 90% are 68.06%, 81.98%, and 96.66%, respectively, showing that our SCS-GMN can perceive changes in similarity thresholds and identify communities with different similarity requirements. In this way, users are allowed to configure their desired community similarity threshold and retrieve communities that meet this similarity criterion by SCS-GMN.

I. Scalability Analysis

Fig. 11 presents the performance (left Y-axis) and training efficiency (right Y-axis) of the improved SCS-GMN on several large-scale graph datasets. For this experiment, we extracted target graphs with scales ranging from millions to tens of millions of nodes from the ogbn-papers100 M dataset. The results show that SCS-GMN achieves an average CSIM of 88.83% and a training time of 0.88 h, demonstrating its strong scalability on large-scale graphs.

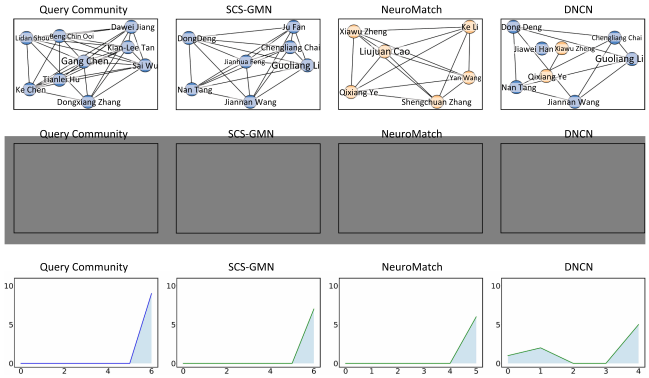


Fig. 12. Case study on DBLP: communities (top), degree distribution (middle), and coreness distribution (bottom).

TABLE XIII
CASE STUDY ON DBLP: DETAIL STATISTICS ON ALL METRICS

Communities →	C_q	SCS-GMN	NeuroMatch	DNCN
Avg. coreness	6	**6 (1)**	5 (2)	2.75 (3)
Size	9	7 (2)	6 (3)	**8 (1)**
Density	0.92	**1 (1)**	**1 (1)**	0.39 (2)
$\texttt{CSIM}()$	-	**96.56 (1)**	90.45 (2)	54.69 (3)

J. Case Studies

To demonstrate the practical applicability of SCS-GMN, we conducted case studies on several commonly used graph datasets, including DBLP, Cora, Citeseer, and Pubmed. Here, we first present the case study on DBLP network to illustrate how SCS-GMN performs in a real-world academic collaboration scenario. Specifically, we extracted several research groups from different domains, including Database (DB), Computer Vision (CV), and Natural Language Processing (NLP) to construct the target graph for training. In the testing phase, we used the well-known DB research group led by Prof. Chen (not included in the target graph) as the query community to present the search results in a real-world scenario. Fig. 12 (top) shows such a query community C_q (left) and three result communities C_r of SCS-GMN and two baselines. We observe that the C_r of SCS-GMN is more structurally similar to C_q than others, and also belongs to the DB research field. In Fig. 12 (middle and bottom), we display the degree and coreness distribution of different C_r, where the X-axis represents the degree or coreness values, and the Y-axis reflects the proportion of nodes having this degree or coreness. The C_r of SCS-GMN exhibits a similar structural distribution to C_q, which also demonstrates its superiority. In contrast, NeuroMatch relies heavily on anchored local subgraph matching. It preserves local structural correspondences around individual nodes, but fails to capture global community-level features. DNCN, which emphasizes node alignment through local neighborhood consistency, returns a community with a similar size. However, due to its limited consideration of structural matching, it may include several unconnected nodes with similar features, e.g., Prof. Han, a well-known researcher in DB. We also summarize the structural statistics of C_q and C_r, including average coreness, density, size, as well as $\texttt{CSIM}$ in Table XIII. These observations indicate that preserving local structural matching or node-level alignment alone is insufficient for similar community search, because the returned communities

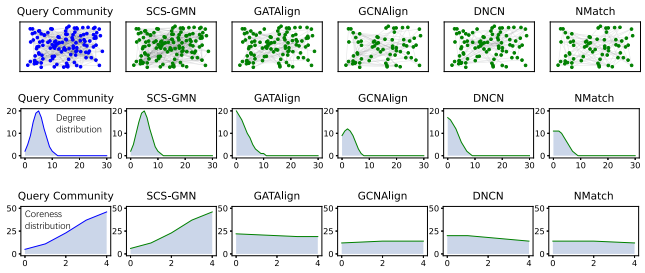


Fig. 13. Case study on Cora: communities (top), degree distribution (middle), and coreness distribution (bottom).

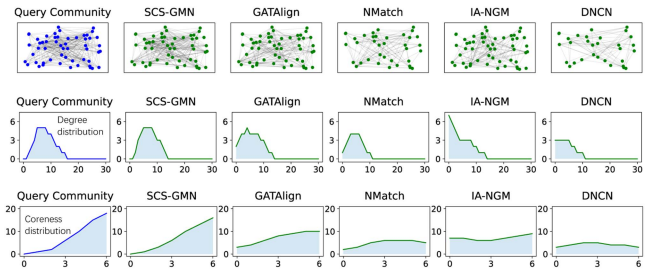


Fig. 14. Case study on Citeseer: communities (top), degree distribution (middle), and coreness distribution (bottom).

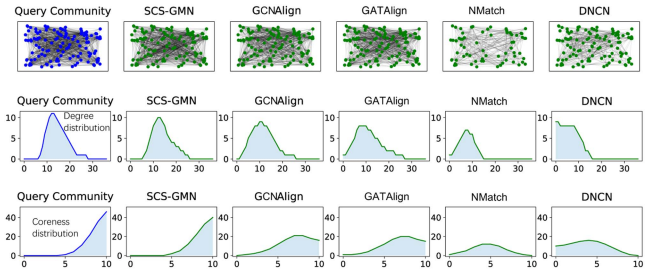


Fig. 15. Case study on Pubmed: communities (top), degree distribution (middle), and coreness distribution (bottom).

TABLE XIV
CASE STUDY ON CORA: DETAIL STATISTICS ON ALL METRICS

Communities →	C_q	SCS-GMN	GATAlign	GCNAlign	DNCN	NeuroMatch
Avg. coreness	4	**3.9 (1)**	2.36 (2)	2.11 (3)	1.56 (5)	1.88 (4)
Size	125	**127 (1)**	70 (3)	70 (3)	90 (2)	68 (4)
Density	0.044	**0.044 (1)**	0.048 (3)	0.045 (2)	0.028 (5)	0.038 (4)
CSIM()	-	**99.95 (1)**	74.56 (2)	70.37 (3)	62.93 (5)	64.47 (4)

TABLE XV
CASE STUDY ON CITESEER: DETAIL STATISTICS ON ALL METRICS

Communities →	C_q	SCS-GMN	GATAlign	NeuroMatch	IA-NGM	DNCN
Avg. coreness	5.48	**5.13 (1)**	3.91 (2)	3.41 (3)	3.24 (4)	2.81 (5)
Size	54	53 (2)	**54 (1)**	37 (3)	**54 (1)**	31 (4)
Density	0.169	**0.158 (1)**	0.118 (4)	0.149 (2)	0.088 (5)	0.146 (3)
CSIM()	-	**99.58 (1)**	89.88 (2)	83.12 (3)	75.21 (4)	69.51 (5)

may still deviate from the query community in key community-level properties, leading to lower overall community similarity.

Following the DBLP case study, we further conduct similar analyses on three datasets: Cora, Citeseer, and Pubmed. Figs. 13–15 presents the visualizations of the result communities for each method, including the community structures, degree distributions, and coreness distributions. Tables XIV-XVI summarizes structural statistics of C_q and C_r, including average coreness, density, and size. We found

TABLE XVI
CASE STUDY ON PUBMED: DETAIL STATISTICS ON ALL METRICS

Communities →	C_q	SCS-GMN	GATAlign	NeuroMatch	IA-NGM	DNCN
Avg. coreness	10.0	**9.59 (1)**	7.19 (2)	6.98 (3)	4.6 (4)	3.59 (5)
Size	123	117 (2)	**123 (1)**	**123 (1)**	68 (4)	110 (3)
Density	0.125	**0.128 (1)**	0.098 (3)	0.097 (4)	0.115 (2)	0.055 (5)
CSIM()	-	**99.77 (1)**	92.83 (2)	91.77 (3)	64.14 (4)	52.40 (5)

that SCS-GMN consistently achieves the best results across all datasets. For instance, in Cora, ours achieved the highest CSIM by matching global statistics almost identically to C_q, whereas baselines deviated significantly in at least one dimension. Even when baselines matched single statistics (e.g., size in Citeseer), they failed to replicate the degree/coreness distributions, leading to inferior overall effectiveness. This advantage comes from its similarity-based self-supervised learning, which introduces community-level guidance into representation learning and enables the model to return a community that is more similar to the query community in multiple aspects.

VI. RELATED WORK

Since similar community search (SCS) is highly related to community search, neural subgraph matching, graph alignment, and graph similarity learning, we review prior studies in these domains to clarify the position of our work.

Community search (CS): Existing CS methods are typically categorized by graph type. First, *CS on homogeneous graphs*. Many works model the cohesive community as k-core [36], [75] and k-truss [38], [39], [40]. To support attributed graphs, various attribute cohesiveness metrics are defined and integrated with structure cohesiveness [39], [76], [77], [78], [79]. Second, *CS on heterogeneous graphs*. The concept of meta-path $\mathcal{P}$ is often used to show the relation between node types, leading to new community models like $(k, \mathcal{P})$-core [6], $(k, \mathcal{P})$-Btruss, and $(k, \mathcal{P})$-Ctruss [80]. Despite their effectiveness, these studies primarily address a node-driven problem: finding a cohesive community that contains a specific query node. However, they overlook scenarios requiring global structure-aware community retrieval, where the goal is to identify communities structurally similar to a given example across the entire graph. This capability is essential for complex applications like social marketing, structural genomics, and criminal group discovery. This gap motivates the SCS study in this paper. Unlike the node-driven CS, SCS adopts a graph-driven paradigm, treating the entire query community as a template to retrieve subgraphs preserving similar macroscopic topological patterns and internal cohesion.

Neural subgraph matching (NSM): Subgraph matching is mainly studied under two classical formulations, namely subgraph isomorphism and subgraph homomorphism [81], [82], [83]. The former focuses on exact edge-preserving one-to-one matching between the query graph and a subgraph of the target graph, while the latter relaxes this requirement by allowing non-injective mappings. The essence of NSM is to predict a matched subgraph based on node matching metrics, where recent methods typically learn node-wise affinities, correspondences, or matching-aware embeddings in a data-driven manner [29], [30], [69], [70], [71], [72], [84], [85]. NeuroMatch [29] derives subgraph relationships from node relationships in the embedding space, Sub-GMN [30] uses neural tensor networks [86] to model node similarity between two graphs. Other works [69], [84] further emphasize isomorphic matching by imposing

connectivity-preserving constraints during learning. Due to the strictness of exact isomorphism, recent studies have relaxed these constraints by learning a node-wise affinity matrix to derive one-to-one node correspondences, without explicitly enforcing exact edge-wise matching. For example, IA-NGM [70] and IA-SSGM [71] iteratively exploit predicted matching results to assist model training. StableGM [72] formulates subgraph matching as a contrastive learning task by introducing hard negative sampling and a contrastive loss function to improve robustness. Despite these advancements, none of these methods provide an effective mechanism for measuring community similarity via community-specific metrics. Consequently, they fail to guide the learning process toward optimizing global community similarity across multiple macroscopic dimensions, such as cohesiveness, size, and density. This motivates us to design a comprehensive, multi-perspective assessment of community similarity.

Graph alignment (GA): GA aims to discover correspondences between two graphs with GNNs being used. [87] reveals the relationship between GCN and consistency-based alignment methods. In [88], both graph and node embeddings are used for multi-graphs alignment. Besides, some works focus on knowledge graph alignment, such as [31] using GCN to learn entity representations for entity-alignment. Other works emphasize the importance of relations during the learning. For example, GAEA [73] combines relational information from neighborhood when aggregating nodes features. DNCN [74] assumes aligned entities have similar neighborhood information, thus using relational alignment to optimize entity alignment. Nevertheless, existing GA methods primarily focus on node-wise correspondences, largely overlooking community-level structural similarity. More precisely, this node-centric perspective lacks an understanding of what structure makes a similar community, preventing it from learning representative features for identifying a similar community. This motivates us to design a cross-graph matching network with similarity-based self-supervised learning, aiming to minimize the discrepancy in macroscopic community features between the result and query community.

Graph similarity learning (GSL): Graph similarity learning studies how to predict a similarity over graph pairs. It is closely related to classical graph similarity computation, where predefined measures such as Graph Edit Distance (GED) and Maximum Common Subgraph (MCS) are used to compare two graphs [23], [89]. However, these classical formulations are often computationally expensive and difficult to scale, which motivates learning-based approaches that estimate graph-pair similarity directly. Existing GSL methods can be roughly grouped into graph embedding-based, GNN-based, and deep graph kernel-based approaches [90]. Representative methods include GMN [28], which introduces cross-graph matching for graph-pair similarity learning, H2MN [87], which performs hierarchical hypergraph matching, CGMN [91], which incorporates contrastive self-supervision into graph matching, and recent studies such as neural graph matching for GED computation [23] and GraSP [92]. However, these approaches lack the capability to inversely leverage predicted graph similarity for graph matching. Therefore, they cannot directly apply to the SCS problem studied in this paper. Nevertheless, GSL provides valuable inspiration: integrating graph similarity learning into the SCS framework could internalize similarity metrics within the model, thereby further enhancing its performance. We leave this promising direction for future investigation.

VII. Conclusion and Discussion

Conclusion: To address the similar community search (SCS) problem, we introduce a unified metric that evaluates community similarity by considering cohesiveness, size, and density. We propose a two-phase SCS-GMN model. In the cross-graph matching phase, a GNN-based graph matching network interacts with structural features between two graphs. In the learning phase, a joint loss combining two similarity-based self-supervised losses guides training towards enhancing community similarity. Experimental results on real-world datasets demonstrate SCS-GMN's superiority.

Discussion: In our implementation, nodes with attributes can be incorporated into the learning process via feature fusion to facilitate semantic alignment. This approach relies on the implicit assumption that the semantics underlying attributes are complementary to (i.e., consistent with) structural features, an assumption that is also consistent with the construction of many benchmark datasets such as those from SNAP, where ground-truth communities are typically annotated by grouping nodes with similar attributes [93]. conversely, inconsistencies between the two may introduce certain adverse effects, potentially causing the failure of SCS. Therefore, it is crucial to address the trade-off between semantics and structure, particularly in more open scenarios where structural similarity does not necessarily imply semantic relatedness. Recent studies, such as DBCD [94], have also highlighted the importance of balancing topology and semantics in community analysis. From this perspective, one possible direction is to extend CSIM by introducing an additional semantic-similarity component when reliable node attributes are available, so that community similarity can be evaluated from both structural and semantic aspects. In parallel, the joint loss in community similarity-guided self-supervised learning could be augmented with a semantic consistency term, enabling the model to preserve not only cohesiveness, size, and density similarities but also community-level semantic alignment. In the future, we will extend SCS-GMN to attributed, weighted heterogeneous graphs by considering the aforementioned balancing issue.

References

[1] D. J. Watts, P. S. Dodds, and M. E. J. Newman, "Identity and search in social networks," *Science*, vol. 296, no. 5571, pp. 1302–1305, 2002.
[2] P. Campana and F. Varese, "Studying organized crime networks: Data sources, boundaries and the limits of structural measures," *Soc. Netw.*, vol. 69, pp. 149–159, 2022.
[3] P. Pesantez-Cabrera and A. Kalyanaraman, "Efficient detection of communities in biological bipartite networks," *IEEE ACM Trans. Comput. Biol. Bioinf.*, vol. 16, no. 1, pp. 258–271, Jan./Feb. 2019.
[4] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, no. 3, pp. 75–174, 2010.
[5] Y. Wang et al., "Scalable community search over large-scale graphs based on graph transformer," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 1680–1690.
[6] Y. Fang, Y. Yang, W. Zhang, X. Lin, and X. Cao, "Effective and efficient community search over large heterogeneous information networks," *Proc. VLDB Endowment*, vol. 13, no. 6, pp. 854–867, 2020.
[7] J. Ye, Y. Zhu, and L. Chen, "Top-r keyword-based community search in attributed graphs," in *Proc. Int. Conf. Data Eng.*, 2023, pp. 1652–1664.
[8] X. Xu, J. Liu, Y. Wang, and X. Ke, "Academic expert finding via (k, p)-core based embedding over heterogeneous graphs," in *Proc. Int. Conf. Data Eng.*, 2022, pp. 338–351.
[9] Y. Wang, C. Gu, X. Xu, X. Zeng, X. Ke, and T. Wu, "Efficient and effective (k, p)-Core-Based community search over attributed heterogeneous information networks," *Inf. Sci.*, vol. 661, 2024, Art. no. 120076.
[10] D. Edge et al., "From local to global: A graph RAG approach to query-focused summarization," 2024, *arXiv:2404.16130*.

[11] B. Peng et al., "Graph retrieval-augmented generation: A survey," 2024, *arXiv:2408.08921*.

[12] X. Gou, W. Zheng, Y. Wang, X. Xu, and Z. Yu, "A comprehensive survey and experimental study of learning-based community search," *Proc. VLDB Endowment*, vol. 18, no. 9, pp. 2941–2954, 2025.

[13] J. Chen, Y. Xia, J. Gao, Z. Li, and H. Chen, "CommunityDF: A guided denoising diffusion approach for community search," in *Proc. 41st IEEE Int. Conf. Data Eng.*, 2025, pp. 460–473.

[14] L. Lin et al., "NCSAC: Effective neural community search via attribute-augmented conductance," *IEEE Trans. Knowl. Data Eng.*, vol. 38, no. 2, pp. 1221–1235, Feb. 2026.

[15] R. Guimerà, L. Danon, A. Díaz-Guilera, F. Giralt, and A. Arenas, "Self-similar community structure in a network of human interactions," *Phys. Rev. E., Stat., Nonlinear, Soft Matter Phys.*, vol. 68, 2002, Art. no. 065103.

[16] D. McKenzie-Mohr, *Fostering Sustainable Behavior: An Introduction to Community-Based Social Marketing*, 3rd ed. Gabriola, BC, Canada: New Society Publishers, 2011.

[17] J.-M. Chandonia and S. E. Brenner, "The impact of structural genomics: Expectations and outcomes," *Science*, vol. 311, no. 5759, pp. 347–351, 2006.

[18] C. Y. Cormier et al., "Protein structure initiative material repository: An open shared public resource of structural genomics plasmids for the biological community," *Nucleic Acids Res.*, vol. 38, pp. D743–D749, 2009.

[19] C. Zhang and S.-H. Kim, "Overview of structural genomics: From structure to function," *Curr. Opin. Chem. Biol.*, vol. 7, no. 1, pp. 28–32, 2003.

[20] H. M. Berman et al., "The protein structure initiative structural genomics knowledgebase," *Nucleic Acids Res.*, vol. 37, pp. D365–D368, 2008.

[21] E. Chattoe and H. Hamill, "It's not who you know—it's what you know about people you don't know that counts: extending the analysis of crime groups as social networks,"*Brit. J. Criminol.*, vol. 45, no. 6, pp. 860–876, 2005.

[22] T. Wang, C. Rudin, D. Wagner, and R. Sevieri, "Learning to detect patterns of crime," in *Proc. Eur. Conf. Mach. Learn. Princ. Pract. Knowl. Discov. Databases*, 2013, pp. 515–530.

[23] C. Piao, T. Xu, X. Sun, Y. Rong, K. Zhao, and H. Cheng, "Computing graph edit distance via neural graph matching," *Proc. VLDB Endowment*, vol. 16, no. 8, pp. 1817–1829, 2023.

[24] F. Zhang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Finding critical users for social network engagement: The collapsed k-core problem," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 245–251.

[25] Y. Fang et al., "A survey of community search over big graphs," *VLDB J.*, vol. 29, no. 1, pp. 353–392, 2020.

[26] H. Xu, D. Luo, H. Zha, and L. Carin, "Gromov-Wasserstein learning for graph matching and node embedding," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6932–6941.

[27] Y. Bai, D. Xu, Y. Sun, and W. Wang, "Detecting small query graphs in a large graph via neural subgraph search," 2022, *arXiv:2207.10305*.

[28] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3835–3845.

[29] R. Ying, Z. Lou, J. You, C. Wen, A. Canedo, and J. Leskovec, "Neural subgraph matching," 2020, *arXiv: 2007.03092*.

[30] Z. Lan, L. Yu, L. Yuan, Z. Wu, Q. Niu, and F. Ma, "Sub-GMN: The neural subgraph matching network model," 2022, *arXiv:2104.00186*.

[31] Z. Wang, Q. Lv, X. Lan, and Y. Zhang, "Cross-lingual knowledge graph alignment via graph convolutional networks," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 349–357.

[32] H. Cheng, D. Luo, and H. Xu, "DHOT-GM: robust graph matching using A differentiable hierarchical optimal transport framework," 2023, *arXiv: 2310.12081*.

[33] J. Tang, W. Zhang, J. Li, K. Zhao, F. Tsung, and J. Li, "Robust attributed graph alignment via joint structure learning and optimal transport," in *Proc. Int. Conf. Data Eng.*, 2023, pp. 1638–1651.

[34] X. Wang, J. Li, L. Yang, H. Mi, and J. Y. Yu, "Weakly-supervised learning for community detection based on graph convolution in attributed networks," *Int. J. Mach. Learn. Cybern.*, vol. 12, pp. 3529–3539, 2021.

[35] I. Bakurov, M. Buzzelli, R. Schettini, M. Castelli, and L. Vanneschi, "Structural similarity index (SSIM) revisited: A data-driven approach," *Expert Syst. Appl.*, vol. 189, 2022, Art. no. 116087.

[36] F. Bonchi, A. Khan, and L. Severini, "Distance-generalized core decomposition," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 1006–1023.

[37] Y. Wang, J. Liu, X. Xu, X. Ke, T. Wu, and X. Gou, "Efficient and effective academic expert finding on heterogeneous graphs through (*k, p*)-core based embedding," *ACM Trans. Knowl. Discov. Data*, vol. 17, no. 6, pp. 85:1–85:35, 2023.

[38] X. Huang, L. V. S. Lakshmanan, J. X. Yu, and H. Cheng, "Approximate closest community search in networks," *Proc. VLDB Endowment*, vol. 9, no. 4, pp. 276–287, 2015.

[39] Q. Liu, Y. Zhu, M. Zhao, X. Huang, J. Xu, and Y. Gao, "VAC: Vertex-centric attributed community search," in *Proc. Int. Conf. Data Eng.*, 2020, pp. 937–948.

[40] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1311–1322.

[41] W. Cui, Y. Xiao, H. Wang, Y. Lu, and W. Wang, "Online search of overlapping communities," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 277–288.

[42] C. E. Tsourakakis, F. Bonchi, A. Gionis, F. Gullo, and M. A. Tsiarli, "Denser than the densest subgraph: extracting optimal quasi-cliques with quality guarantees," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2013, pp. 104–112.

[43] J. Hu, X. Wu, R. Cheng, S. Luo, and Y. Fang, "Querying minimal steiner maximum-connected subgraphs in large graphs," in *Proc. Conf. Inf. Knowl. Manage.*, 2016, pp. 1241–1250.

[44] Y. Wang et al., "Scalable community search with accuracy guarantee on attributed graphs," in *Proc. IEEE 40th Int. Conf. Data Eng.*, 2024, pp. 2737–2750.

[45] L. Sun, X. Huang, R. Li, B. Choi, and J. Xu, "Index-based intimate-core community search in large weighted graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 9, pp. 4313–4327, Sep. 2022.

[46] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," in *Proc. IEEE Int. Conf. Data Mining*, 2012, pp. 745–754.

[47] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–106, 2008.

[48] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 4292–4293.

[49] G. M. Namata, B. London, L. Getoor, and B. Huang, "Query-driven active surveying for collective classification," in *Proc. 10th Int. Workshop Mining Learn. Graphs*, vol. 8, 2012, p. 1.

[50] B. Rozemberczki and R. Sarkar, "Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models," in *Proc. Conf. Inf. Knowl. Manage.*, 2020, pp. 1325–1334.

[51] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale attributed node embedding," *J. Complex Netw.*, vol. 9, no. 2, 2021.

[52] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[53] C. Godard, O. M. Aodha, and G. J. Brostow, "Unsupervised monocular depth estimation with left-right consistency," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6602–6611.

[54] C. Godard, O. M. Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3827–3837.

[55] K. Yao and L. Chang, "Efficient size-bounded community search over large networks," *Proc. VLDB Endowment*, vol. 14, no. 8, pp. 1441–1453, 2021.

[56] F. Radicchi, C. Castellano, F. Cecconi, V. Loreto, and D. Parisi, "Defining and identifying communities in networks," *Proc. Nat. Acad. Sci.*, vol. 101, no. 9, pp. 2658–2663, 2004.

[57] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017.

[58] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.

[59] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv: 1710.10903*.

[60] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[61] V. Batagelj and M. Zaversnik, "An o (m) algorithm for cores decomposition of networks," 2003, *arXiv:cs/0310049*.

[62] Q. Zhang, Z. Zhao, H. Zhou, X. Li, and C. Li, "Self-supervised contrastive learning on heterogeneous graphs with mutual constraints of structure and feature," *Inf. Sci.*, vol. 640, 2023, Art. no. 119026.

[63] W. Khaouid, M. Barsky, S. Venkatesh, and A. Thomo, "*K*-core decomposition of large networks on a single PC," *Proc. VLDB Endow.*, vol. 9, no. 1, pp. 13–23, 2015.

[64] K. Shin, T. Eliassi-Rad, and C. Faloutsos, "Patterns and anomalies in k-cores of real-world graphs with applications," *Knowl. Inf. Syst.*, vol. 54, no. 3, pp. 677–710, 2018.
[65] C. Ying et al., "Do transformers really perform badly for graph representation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 28877–28888.
[66] K. Wang, Z. Shen, C. Huang, C.-H. Wu, Y. Dong, and A. Kanakia, "Microsoft academic graph: When experts are not enough," *Quantitative Sci. Stud.*, vol. 1, no. 1, pp. 396–413, 2020.
[67] Code and datasets, "Code and datasets," 2024. [Online]. Available: https://anonymous.4open.science/r/SCS-GMN-E584/
[68] "DBLP," 2022. [Online]. Available: http://dblp.uni-trier.de/xml/
[69] Z. Lan, Y. Ma, L. Yu, L. Yuan, and F. Ma, "AEDNet: Adaptive edge-deleting network for subgraph matching," *Pattern Recognit.*, vol. 133, 2023, Art. no. 109033.
[70] T. Qin, S. Tu, and L. Xu, "IA-NGM: A bidirectional learning method for neural graph matching with feature fusion," *Mach. Learn.*, vol. 113, no. 4, pp. 1743–1769, 2024.
[71] W. Guo, L. Zhang, S. Tu, and L. Xu, "Self-supervised bidirectional learning for graph matching," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2023, pp. 7784–7792.
[72] G. Ratnayaka, Q. Wang, and Y. Li, "Contrastive learning for supervised graph matching," in *Uncertainty Artif. Intell.*, vol. 216, 2023, pp. 1718–1729.
[73] F. Xie, X. Zeng, B. Zhou, and Y. Tan, "Improving knowledge graph entity alignment with graph augmentation," in *Proc. Pacific-Asia Conf. Knowl. Discov. Data Mining*, 2023, pp. 3–14.
[74] Y. Zhang, Y. Li, X. Wei, Y. Yang, L. Liu, and Y. L. Murphey, "Graph matching for knowledge graph alignment using edge-coloring propagation," *Pattern Recognit.*, vol. 144, 2023, Art. no. 109851.
[75] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu, "Effective and efficient community search over large directed graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2093–2107, vol. 31, no. 11, pp. 2093–2107, Nov. 2019.
[76] L. Chen, C. Liu, K. Liao, J. Li, and R. Zhou, "Contextual community search over large social networks," in *Proc. Int. Conf. Data Eng.*, 2019, pp. 88–99.
[77] Y. Fang, R. Cheng, X. Li, S. Luo, and J. Hu, "Effective community search over large spatial graphs," *Proc. VLDB Endowment*, vol. 10, no. 6, pp. 709–720, 2017.
[78] X. Huang and L. V. S. Lakshmanan, "Attribute-driven community search," *Proc. VLDB Endowment*, vol. 10, no. 9, pp. 949–960, 2017.
[79] Z. Zhang, X. Huang, J. Xu, B. Choi, and Z. Shang, "Keyword-centric community search," in *Proc. Int. Conf. Data Eng.*, 2019, pp. 422–433.
[80] Y. Yang, Y. Fang, X. Lin, and W. Zhang, "Effective and efficient truss computation over large heterogeneous information networks," in *Proc. Int. Conf. Data Eng.*, 2020, pp. 901–912.
[81] J. R. Ullmann, "An algorithm for subgraph isomorphism," *J. ACM*, vol. 23, no. 1, pp. 31–42, 1976.
[82] W. Fan, J. Li, S. Ma, H. Wang, and Y. Wu, "Graph homomorphism revisited for graph matching," *Proc. VLDB Endowment*, vol. 3, no. 1, pp. 1161–1172, 2010. [Online]. Available: https://vldb.org/pvldb/vol3/R103.pdf
[83] X. Sun, S. Sun, Q. Luo, and B. He, "An in-depth study of continuous subgraph matching," *Proc. VLDB Endowment*, vol. 15, no. 7, pp. 1403–1416, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p1403-sun.pdf
[84] X. Liu and Y. Song, "Graph convolutional networks with dual message passing for subgraph isomorphism counting and matching," in *Proc. Conf. Assoc. Adv. Artif. Intell.*, 2022, pp. 7594–7602.
[85] J. Bo and Y. Fang, "Contrastive general graph matching with adaptive augmentation sampling," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, Jeju, Korea, 2024, pp. 3724–3732. [Online]. Available: https://www.ijcai.org/proceedings/2024/412
[86] R. Socher, D. Chen, C. D. Manning, and A. Y. Ng, "Reasoning with neural tensor networks for knowledge base completion," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2013, pp. 926–934.
[87] S. Zhang, H. Tong, L. Jin, Y. Xia, and Y. Guo, "Balancing consistency and disparity in network alignment," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 2212–2222.
[88] X. Chu, X. Fan, D. Yao, Z. Zhu, J. Huang, and J. Bi, "Cross-network embedding for multi-network alignment," in in *Proc. Int. Conf. World Wide Web*, 2019, pp. 273–284.
[89] H. Bunke and K. Shearer, "A graph distance metric based on the maximal common subgraph," *Pattern Recognit. Lett.*, vol. 19, no. 3–4, pp. 255–259, 1998.
[90] G. Ma, N. K. Ahmed, T. L. Willke, and P. S. Yu, "Deep graph similarity learning: A survey," *Data Mining Knowl. Discov.*, vol. 35, pp. 688–725, 2021.
[91] D. Jin et al., "CGMN: A contrastive graph matching network for self-supervised graph similarity learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2101–2107.
[92] H. Zheng, J. Shi, and R. Yang, "GraSP: Simple yet effective graph similarity predictions," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 22884–22892.
[93] J. Leskovec and A. Krevl, "SNAP datasets: Stanford large network dataset collection," 2014. [Online]. Available: http://snap.stanford.edu/data
[94] Y. Wang, Y. Liu, X. Sun, and J. Fu, "DBCD: Deep balanced community detection via consensus-guided joint optimization in attributed networks," *Expert Syst. Appl.*, 2025, Art. no. 129487.